Project Structure

File: ./traffic/traffic_generator.py

```python
#!/usr/bin/env python3
import os
import time
import argparse

def generate_normal_traffic():
    """Generate steady baseline traffic"""
    print("  Generating NORMAL traffic pattern...")
    for i in range(1, 17):
        for j in range(1, 17):
            if i != j:
                os.system(f'sudo mnexec -a $(pgrep -f "mininet:h{i}") ping -c 5 10.0.0.{j} >
                time.sleep(0.1)

def generate_spike_traffic():
    """Generate sudden traffic spike on specific switches"""
    print("  Generating SPIKE traffic pattern (targeting Slave1 switches)...")
    # Heavy traffic on h1-h8 (connected to s1-s4, managed by Slave1)
    for i in range(1, 9):
        for j in range(1, 9):
            if i != j:
                os.system(f'sudo mnexec -a $(pgrep -f "mininet:h{i}") ping -c 50 -f 10.0.0.{j

    print("  Traffic spike initiated! Check Master Controller logs for evocation...")

def generate_gradual_traffic():
    """Gradually increase traffic load"""
    print("  Generating GRADUAL traffic increase...")
    for intensity in range(1, 10):
        print(f"   Intensity level: {intensity}/10")
        for i in range(1, 9):
            for _ in range(intensity):
                j = (i % 8) + 1
                os.system(f'sudo mnexec -a $(pgrep -f "mininet:h{i}") ping -c 10 10.0.0.{j} >
        time.sleep(5)

def stop_traffic():
    """Kill all ping processes"""
    print("  Stopping all traffic...")
    os.system('sudo killall ping > /dev/null 2>&1')
```

```
if __name__ == '__main__':
   parser = argparse.ArgumentParser(description='SDN Traffic Generator')
   parser.add_argument('pattern', choices=['normal', 'spike', 'gradual', 'stop'],
                       help='Traffic pattern to generate')
   args = parser.parse_args()

   if args.pattern == 'normal':
       generate_normal_traffic()
   elif args.pattern == 'spike':
       generate_spike_traffic()
   elif args.pattern == 'gradual':
       generate_gradual_traffic()
   elif args.pattern == 'stop':
       stop_traffic()
-e
```

File: ./traffic/aggressive_traffic.py

```
#!/usr/bin/env python3
import os
import time

print("  Starting AGGRESSIVE traffic generation...")
print("This will flood h1-h8 with heavy ping traffic")

# Generate flood pings between hosts 1-8 (connected to slave1's switches)
commands = []
for i in range(1, 9):
   for j in range(1, 9):
       if i != j:
           # -f = flood mode, -c 1000 = 1000 packets, -s 1024 = 1KB packet size
           cmd = f'sudo mnexec -a $(pgrep -f "mininet:h{i}") ping -f -c 1000 -s 1024 10.0.0.
           commands.append(cmd)

print(f"Launching {len(commands)} concurrent ping floods...")

for cmd in commands:
   os.system(cmd)
   time.sleep(0.05)  # Small delay to stagger launches

print("  Traffic generation started!")
print("Check Master Controller logs for overload detection...")
print("This will run for about 20-30 seconds")
-e
```

File: ./agents/lstm_extractor.py

```python
#!/usr/bin/env python3
"""
LSTM Feature Extractor for Temporal Awareness
Integrates with Stable-Baselines3 DQN
"""
import torch
import torch.nn as nn
from stable_baselines3.common.torch_layers import BaseFeaturesExtractor
import gymnasium as gym

class LSTMFeatureExtractor(BaseFeaturesExtractor):
    """
    Custom LSTM feature extractor for temporal state representation

    This adds proactive capability by learning from historical patterns
    """
    def __init__(self, observation_space: gym.spaces.Box,
                 features_dim: int = 128,
                 lstm_hidden: int = 64,
                 lstm_layers: int = 2):
        super().__init__(observation_space, features_dim)

        n_input = observation_space.shape[0]

        # LSTM for processing temporal sequences
        self.lstm = nn.LSTM(
            input_size=n_input,
            hidden_size=lstm_hidden,
            num_layers=lstm_layers,
            batch_first=True,
            dropout=0.1 if lstm_layers > 1 else 0
        )

        # Fully connected layers after LSTM
        self.fc = nn.Sequential(
            nn.Linear(lstm_hidden, features_dim),
            nn.ReLU(),
            nn.LayerNorm(features_dim)
        )

    def forward(self, observations: torch.Tensor) -> torch.Tensor:
        """
        Forward pass through LSTM
```

```
        Args:
            observations: (batch_size, obs_dim) tensor

        Returns:
            features: (batch_size, features_dim) tensor
        """
        # Reshape for LSTM: (batch, seq_len=1, input_size)
        # In training, obs already contains history concatenated
        # So we treat it as a single timestep with rich features
        batch_size = observations.shape[0]

        # Add sequence dimension
        lstm_input = observations.unsqueeze(1)  # (batch, 1, obs_dim)

        # Pass through LSTM
        lstm_out, (hidden, cell) = self.lstm(lstm_input)

        # Take the last output
        last_output = lstm_out[:, -1, :]  # (batch, lstm_hidden)

        # Pass through FC layers
        features = self.fc(last_output)

        return features


class EnhancedLSTMExtractor(BaseFeaturesExtractor):
    """
    Enhanced LSTM with attention mechanism for better temporal modeling
    """
    def __init__(self, observation_space: gym.spaces.Box,
                 features_dim: int = 128,
                 lstm_hidden: int = 64,
                 history_length: int = 5,
                 state_dim: int = 17):
        super().__init__(observation_space, features_dim)

        self.history_length = history_length
        self.state_dim = state_dim

        # LSTM for sequential processing
        self.lstm = nn.LSTM(
            input_size=state_dim,
            hidden_size=lstm_hidden,
            num_layers=2,
            batch_first=True,
```

```
            dropout=0.1
        )

        # Attention mechanism
        self.attention = nn.Linear(lstm_hidden, 1)

        # Output layers
        self.fc = nn.Sequential(
            nn.Linear(lstm_hidden, features_dim),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(features_dim, features_dim),
            nn.ReLU()
        )

    def forward(self, observations: torch.Tensor) -> torch.Tensor:
        """
        Forward pass with attention
        """
        batch_size = observations.shape[0]

        # Reshape observations into sequence
        # observations: (batch, state_dim * (history_length + 1))
        # Reshape to: (batch, history_length + 1, state_dim)
        obs_reshaped = observations.view(batch_size, self.history_length + 1, self.state_dim)

        # Pass through LSTM
        lstm_out, _ = self.lstm(obs_reshaped)  # (batch, seq_len, hidden)

        # Apply attention
        attention_weights = torch.softmax(self.attention(lstm_out), dim=1)  # (batch, seq_len
        context = torch.sum(attention_weights * lstm_out, dim=1)  # (batch, hidden)

        # Final features
        features = self.fc(context)

        return features
-e
```

File: ./parked_controller.py

```
from os_ken.base import app_manager
from os_ken.controller import ofp_event
from os_ken.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER
from os_ken.controller.handler import set_ev_cls
from os_ken.ofproto import ofproto_v1_3
```

```python
from os_ken.app.wsgi import WSGIApplication, ControllerBase, route
from webob import Response
import json
import os

parked_instance_name = 'parked_api_app'

class ParkedController(app_manager.OSKenApp):
   OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
   _CONTEXTS = {'wsgi': WSGIApplication}

   def __init__(self, *args, **kwargs):
       super(ParkedController, self).__init__(*args, **kwargs)

       # Get port from environment
       wsgi_port = int(os.environ.get('WSGI_PORT', 8083))

       # Configure WSGI
       wsgi = kwargs['wsgi']
       wsgi.start(
           host='127.0.0.1',
           port=wsgi_port
       )

       self.status = 'parked'
       self.datapaths = {}

       # Register REST API
       wsgi.register(ParkedControlController, {parked_instance_name: self})

       self.logger.info("="*50)
       self.logger.info(f"PARKED CONTROLLER INITIALIZED (SLEEP MODE) on port {wsgi_port}")
       self.logger.info("Waiting to be evoked...")
       self.logger.info("="*50)

   @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
   def switch_features_handler(self, ev):
       """Handle switch connection after being evoked"""
       if self.status == 'active':
           datapath = ev.msg.datapath
           self.datapaths[datapath.id] = datapath
           self.logger.info(f"  Parked controller now managing switch: {datapath.id}")

           ofproto = datapath.ofproto
           parser = datapath.ofproto_parser
           match = parser.OFPMatch()
```

```
            actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                              ofproto.OFPCML_NO_BUFFER)]
            self.add_flow(datapath, 0, match, actions)

    def add_flow(self, datapath, priority, match, actions):
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

    def evoke(self):
        """Activate this controller"""
        self.status = 'active'
        self.logger.info("  CONTROLLER EVOKED - NOW ACTIVE!")
        return True

    def hibernate(self):
        """Put controller back to sleep"""
        self.status = 'parked'
        self.datapaths.clear()
        self.logger.info("  CONTROLLER HIBERNATED")
        return True


class ParkedControlController(ControllerBase):
    def __init__(self, req, link, data, **config):
        super(ParkedControlController, self).__init__(req, link, data, **config)
        self.parked_app = data[parked_instance_name]

    @route('control', '/control/evoke', methods=['POST'])
    def evoke_controller(self, req, **kwargs):
        """REST API to wake up the controller"""
        success = self.parked_app.evoke()
        body = json.dumps({'evoked': success, 'status': self.parked_app.status})
        return Response(content_type='application/json', body=body)

    @route('control', '/control/hibernate', methods=['POST'])
    def hibernate_controller(self, req, **kwargs):
        """REST API to put controller to sleep"""
        success = self.parked_app.hibernate()
        body = json.dumps({'hibernated': success, 'status': self.parked_app.status})
        return Response(content_type='application/json', body=body)
-e
```

File: ./topologies/sdn_topology.py

```python
#!/usr/bin/env python3
from mininet.net import Mininet
from mininet.node import RemoteController, OVSSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel, info
from mininet.link import TCLink

def create_proactive_topology():
    """
    Create SDN topology with multiple controllers
    - Master controller: monitors (port 6633)
    - Slave1: manages switches s1-s4 (port 6634)
    - Slave2: manages switches s5-s8 (port 6635)
    - Parked: standby controller (port 6636)
    """

    net = Mininet(controller=RemoteController,
                  switch=OVSSwitch,
                  link=TCLink,
                  autoSetMacs=True)

    info('*** Adding controllers\n')
    # Master controller (monitoring only, doesn't manage switches directly)
    c0 = net.addController('c0', controller=RemoteController,
                           ip='127.0.0.1', port=6633)

    # Slave controllers
    c1 = net.addController('c1', controller=RemoteController,
                           ip='127.0.0.1', port=6634)
    c2 = net.addController('c2', controller=RemoteController,
                           ip='127.0.0.1', port=6635)

    # Parked controller (not assigned switches initially)
    c3 = net.addController('c3', controller=RemoteController,
                           ip='127.0.0.1', port=6636)

    info('*** Adding switches\n')
    switches = []
    for i in range(1, 9):
        s = net.addSwitch(f's{i}', protocols='OpenFlow13')
        switches.append(s)

    info('*** Adding hosts\n')
    hosts = []
```

```
    for i in range(1, 17):
        h = net.addHost(f'h{i}')
        hosts.append(h)

    info('*** Creating links\n')
    # Connect 2 hosts to each switch
    for i, switch in enumerate(switches):
        net.addLink(hosts[i*2], switch)
        net.addLink(hosts[i*2+1], switch)

    # Inter-switch links (linear topology for simplicity)
    for i in range(len(switches)-1):
        net.addLink(switches[i], switches[i+1])

    info('*** Starting network\n')
    net.build()

    # Start controllers
    c0.start()
    c1.start()
    c2.start()
    c3.start()

    info('*** Assigning switches to controllers\n')
    # Slave1 manages s1-s4
    for i in range(4):
        switches[i].start([c1])
        info(f's{i+1} -> Slave1 (c1)\n')

    # Slave2 manages s5-s8
    for i in range(4, 8):
        switches[i].start([c2])
        info(f's{i+1} -> Slave2 (c2)\n')

    info('*** Network ready!\n')
    info('*** Run traffic generator to test proactive load balancing\n')

    CLI(net)
    net.stop()

if __name__ == '__main__':
    setLogLevel('info')
    create_proactive_topology()
-e
```

File: ./environments/proactive_sdn_env.py

```python
#!/usr/bin/env python3
"""
Proactive SDN Load Balancing Environment - FULLY FIXED VERSION
Agent-driven parking with corrected positive reward structure
"""
import gymnasium as gym
from gymnasium import spaces
import numpy as np
from collections import deque

class ProactiveSDNEnv(gym.Env):
    """
    Enhanced SDN Environment with agent-driven proactive parking
    FIXED: Positive reward baseline with meaningful bonuses/penalties
    """
    metadata = {'render_modes': ['human']}

    def __init__(self,
                 num_active_controllers=3,
                 num_parked_controllers=2,
                 num_switches=10,
                 history_length=5):
        super().__init__()

        self.num_active = num_active_controllers
        self.num_parked = num_parked_controllers
        self.num_switches = num_switches
        self.total_controllers = num_active_controllers + num_parked_controllers
        self.history_length = history_length

        # Action space: do_nothing, migrate (switches x controllers), evoke (2), park (1)
        self.action_space = spaces.Discrete(
            4 + num_switches * num_active_controllers
        )

        # Observation space: controller loads + switch mappings + parked status + latency + c
        self.single_state_dim = (num_active_controllers + num_switches +
                                 num_parked_controllers + 2)
        self.obs_dim = self.single_state_dim * (history_length + 1)

        self.observation_space = spaces.Box(
            low=0, high=1,
            shape=(self.obs_dim,),
            dtype=np.float32
```

```
        )

        # History buffer for LSTM temporal learning
        self.state_history = deque(maxlen=history_length)

        # Energy constants (Watts)
        self.ACTIVE_POWER = 150
        self.PARKED_POWER = 10

        # Simulation parameters
        self.time_step = 0
        self.max_steps = 1000
        self.traffic_phase = 0

        # Random number generator
        self._np_random = None

        # Initialize
        self.reset()

    def reset(self, seed=None, options=None):
        """Reset with imbalanced state and varying controller status"""
        if seed is not None:
            np.random.seed(seed)
            self._np_random = np.random.RandomState(seed)

        # Create imbalanced initial loads
        if np.random.rand() > 0.5:
            self.controller_loads = np.array([
                0.2 + np.random.rand() * 0.2,  # 0.2-0.4
                0.4 + np.random.rand() * 0.2,  # 0.4-0.6
                0.7 + np.random.rand() * 0.2   # 0.7-0.9
            ])
        else:
            imbalance = np.random.rand() * 0.5
            self.controller_loads = np.array([
                0.3 + imbalance,
                0.5,
                0.7 - imbalance
            ])

        self.controller_loads = np.clip(self.controller_loads, 0.1, 0.9)

        # Create matching switch mappings
        num_switches_per_controller = (self.controller_loads * self.num_switches).astype(int)
        diff = self.num_switches - num_switches_per_controller.sum()
```

```
        if diff != 0:
            num_switches_per_controller[0] += diff

        num_switches_per_controller = np.clip(num_switches_per_controller, 0, self.num_switch

        mappings = []
        for controller_id, count in enumerate(num_switches_per_controller):
            mappings.extend([controller_id] * int(count))

        if len(mappings) < self.num_switches:
            mappings.extend([0] * (self.num_switches - len(mappings)))
        elif len(mappings) > self.num_switches:
            mappings = mappings[:self.num_switches]

        self.switch_mappings = np.array(mappings)
        np.random.shuffle(self.switch_mappings)
        self._update_loads()

        # Random starting hour (0-23)
        self.traffic_phase = np.random.randint(0, 24)

        # Start with varying controller states
        # 70% chance to have 1 controller evoked at start
        if np.random.rand() > 0.3:
            self.parked_status = np.array([1, 0])  # One evoked
        else:
            self.parked_status = np.array([0, 0])  # Both parked

        self.current_latency = self._calculate_latency()
        self.current_energy = self._calculate_energy()

        self.state_history.clear()
        current_state = self._get_current_state()
        for _ in range(self.history_length):
            self.state_history.append(current_state)

        self.time_step = 0

        return self._get_observation(), {}

    def _simulate_traffic_variation(self):
        """
        TIME-BASED traffic simulation
        Creates predictable patterns for LSTM to learn
        """
        hour = (self.time_step // 40) % 24
```

```python
        # Strong traffic variation for clear patterns
        if 8 <= hour < 18:  # Business hours: HIGH traffic
            traffic_multiplier = 1.6
        elif 0 <= hour < 6 or 22 <= hour < 24:  # Night: LOW traffic
            traffic_multiplier = 0.3
        else:  # Transition hours
            traffic_multiplier = 1.0

        # Apply with realistic noise
        noise = np.random.uniform(0.85, 1.15, self.num_active)
        self.controller_loads = self.controller_loads * traffic_multiplier * noise

        # Occasional traffic spikes (3% chance)
        if np.random.random() < 0.03:
            spike_controller = np.random.randint(0, self.num_active)
            self.controller_loads[spike_controller] *= 1.5

        self.controller_loads = np.clip(self.controller_loads, 0.05, 0.95)

    def _get_current_state(self):
        """Get current state with time information"""
        hour_normalized = ((self.time_step // 40) % 24) / 24.0

        return np.concatenate([
            self.controller_loads,
            self.switch_mappings / self.num_active,
            self.parked_status,
            [self.current_latency / 100],
            [self.current_energy / 1000]
        ])

    def _get_observation(self):
        """Get full observation including history for LSTM"""
        current = self._get_current_state()
        obs = np.concatenate([current] + list(self.state_history))
        return obs.astype(np.float32)

    def step(self, action):
        """Execute action with time-based traffic simulation"""
        action_type, entity_id, target_id = self._decode_action(action)
        action_success = self._execute_action(action_type, entity_id, target_id)

        self.time_step += 1

        # Apply traffic variation every step
```

```python
        self._simulate_traffic_variation()

        self.current_latency = self._calculate_latency()
        self.current_energy = self._calculate_energy()

        current_state = self._get_current_state()
        self.state_history.append(current_state)

        reward = self._calculate_reward(action_success, action_type)

        done = self.time_step >= self.max_steps
        terminated = done
        truncated = False

        info = {
            'latency': self.current_latency,
            'energy': self.current_energy,
            'load_variance': np.var(self.controller_loads),
            'active_controllers': self.num_active + np.sum(self.parked_status),
            'action_type': action_type,
            'action_success': action_success,
            'hour_of_day': (self.time_step // 40) % 24
        }

        return self._get_observation(), reward, terminated, truncated, info

    def _decode_action(self, action):
        """Decode action with bounds checking"""
        if action == 0:
            return 0, 0, 0  # do_nothing
        elif action == 1:
            return 2, 0, 0  # Evoke parked controller 0
        elif action == 2:
            return 2, 1, 0  # Evoke parked controller 1
        elif action == 3:
            return 3, 0, 0  # Park a controller
        else:
            # Migration action
            migration_action = action - 4
            switch_id = migration_action // self.num_active
            target_controller = migration_action % self.num_active

            if switch_id >= self.num_switches:
                return 0, 0, 0  # Invalid action, convert to do_nothing

            return 1, switch_id, target_controller
```

```python
def _execute_action(self, action_type, entity_id, target_id):
    """Execute decoded action"""
    if action_type == 0:
        return True  # do_nothing always succeeds
    elif action_type == 1:
        return self._migrate_switch(entity_id, target_id)
    elif action_type == 2:
        return self._evoke_controller(entity_id)
    elif action_type == 3:
        return self._park_controller()
    return False

def _migrate_switch(self, switch_id, target_controller):
    """Migrate switch to target controller"""
    if switch_id >= self.num_switches or target_controller >= self.num_active:
        return False

    old_controller = self.switch_mappings[switch_id]
    if old_controller == target_controller:
        return False  # Already assigned

    self.switch_mappings[switch_id] = target_controller
    self._update_loads()
    return True

def _evoke_controller(self, parked_id):
    """
    AGENT-DRIVEN EVOKE: No autonomous time-based logic
    Environment simply executes the agent's decision
    Agent must learn WHEN to evoke through reward signals
    """
    if parked_id >= self.num_parked:
        return False

    if self.parked_status[parked_id] == 1:
        return False  # Already active

    #  FIXED: Just activate, no autonomous decision making
    self.parked_status[parked_id] = 1

    # Log for debugging (helpful for research analysis)
    hour = (self.time_step // 40) % 24
    max_load = np.max(self.controller_loads)
    avg_load = np.mean(self.controller_loads)
```

```
        # Determine why this might be good/bad (for logging only)
        if (8 <= hour < 18) and (max_load > 0.6 or avg_load > 0.45):
            print(f"   EVOKED controller {parked_id} [Day proactive | Hour: {hour:02d}, Max:
        elif max_load > 0.8:
            print(f"   EMERGENCY EVOKE controller {parked_id} [SPIKE | Hour: {hour:02d}, Loa
        else:
            print(f"   EVOKED controller {parked_id} [Hour: {hour:02d}, Max: {max_load:.2f}]

        return True

    def _park_controller(self):
        """
        AGENT-DRIVEN PARK: No autonomous time-based logic
        Environment simply executes the agent's decision
        Agent must learn WHEN to park through reward signals
        """
        active_parked = np.where(self.parked_status == 1)[0]
        if len(active_parked) == 0:
            return False  # Nothing to park

        #  FIXED: Just park, no autonomous decision making
        park_id = active_parked[0]
        self.parked_status[park_id] = 0

        # Log for debugging
        hour = (self.time_step // 40) % 24
        avg_load = np.mean(self.controller_loads)
        max_load = np.max(self.controller_loads)

        # Determine why this might be good/bad (for logging only)
        if (0 <= hour < 6 or 22 <= hour < 24) and avg_load < 0.5:
            print(f"   PARKED controller {park_id} [Night | Hour: {hour:02d}, Avg: {avg_load
        elif (6 <= hour < 22) and avg_load < 0.35 and max_load < 0.5:
            print(f"   PARKED controller {park_id} [Day LOW traffic | Hour: {hour:02d}, Avg:
        else:
            print(f"   PARKED controller {park_id} [Hour: {hour:02d}, Avg: {avg_load:.2f}]")

        self._update_loads()
        return True

    def _update_loads(self):
        """Update controller loads based on switch assignments"""
        for controller_id in range(self.num_active):
            num_switches = np.sum(self.switch_mappings == controller_id)
            self.controller_loads[controller_id] = num_switches / self.num_switches
```

```python
def _calculate_latency(self):
    """
    Calculate network latency with penalties for:
    - Load imbalance (variance)
    - Controller overload (> 0.8)
    """
    base_latency = 10
    load_variance = np.var(self.controller_loads)
    imbalance_penalty = load_variance * 50
    overload_penalty = np.sum(np.maximum(0, self.controller_loads - 0.8)) * 30
    return base_latency + imbalance_penalty + overload_penalty

def _calculate_energy(self):
    """Calculate total energy consumption"""
    num_active = self.num_active + np.sum(self.parked_status)
    active_energy = num_active * self.ACTIVE_POWER
    num_parked = self.num_parked - np.sum(self.parked_status)
    parked_energy = num_parked * self.PARKED_POWER
    return active_energy + parked_energy

def _calculate_reward(self, action_success, action_type):
    """
    FIXED REWARD FUNCTION with positive baseline and meaningful bonuses
    Agent learns optimal parking/evoking patterns from these signals
    """
    # Get context first
    hour = (self.time_step // 40) % 24
    active_count = self.num_active + np.sum(self.parked_status)
    parked_count = self.total_controllers - active_count
    avg_load = np.mean(self.controller_loads)
    max_load = np.max(self.controller_loads)
    load_variance = np.var(self.controller_loads)

    # ==================================================================
    # BASE REWARD: Start with positive value
    # ==================================================================

    reward = 100.0  # Everyone starts positive!

    # ==================================================================
    # PENALTIES (subtract from baseline)
    # ==================================================================

    # Latency penalty (0-30 points)
    latency_penalty = (self.current_latency - 10) * 2.0  # 10ms = baseline
    reward -= max(0, latency_penalty)
```

```python
# Load imbalance penalty (0-40 points)
load_imbalance = max_load - np.min(self.controller_loads)
if load_imbalance > 0.3:
    reward -= load_imbalance * 100

# Overload penalty (0-50 points)
if max_load > 0.8:
    overload_severity = (max_load - 0.8) * 250
    reward -= overload_severity

# Energy penalty based on waste
baseline_energy = self.total_controllers * self.ACTIVE_POWER  # 750W
min_needed_energy = self.num_active * self.ACTIVE_POWER + 20  # Minimum needed
energy_waste = self.current_energy - min_needed_energy
energy_penalty = max(0, energy_waste * 0.15)
reward -= energy_penalty

# ==================================================================
# TIME-BASED BONUSES (add to reward)
# ==================================================================

# NIGHT PARKING BONUS (0-300 points)
if (0 <= hour < 6 or 22 <= hour < 24):
    if parked_count > 0:
        # Base bonus for having controllers parked at night
        night_parking_bonus = parked_count * 150.0

        # Extra bonus if load is actually low (good decision!)
        if avg_load < 0.5:
            night_parking_bonus += parked_count * 100.0

        reward += night_parking_bonus

        # Action bonus for just parking
        if action_type == 3 and action_success:
            reward += 80.0

    # Penalty for NOT parking when you should
    elif parked_count == 0 and avg_load < 0.3:
        reward -= 100.0  # Wasting energy!

# DAY CAPACITY BONUS (0-100 points)
elif (8 <= hour < 18):
    if parked_count == 0:
        # Good! All active during business hours
```

```
        if max_load > 0.5:
            reward += 60.0
    else:
        # Controllers parked during day
        if max_load > 0.7:
            # Very bad! Parked during high load
            reward -= 150.0
        elif max_load < 0.35 and avg_load < 0.3:
            # Good! Adaptive day parking during unusual low traffic
            day_parking_bonus = parked_count * 80.0
            reward += day_parking_bonus
            if action_type == 3 and action_success:
                reward += 50.0

# EMERGENCY HANDLING BONUS (0-150 points)
if max_load > 0.8:
    if parked_count == 0:
        # Good! Have capacity for emergency
        reward += 100.0
        if action_type == 2 and action_success:
            # Just evoked during emergency!
            reward += 70.0
    else:
        # Bad! Parked during overload
        reward -= 120.0

# ==================================================================
# ACTION BONUSES/PENALTIES
# ==================================================================

# Successful migration bonus
if action_type == 1 and action_success:
    # Reward if it improved balance
    if load_imbalance < 0.3:
        reward += 5.0

# Failed action penalty
if not action_success and action_type != 0:
    reward -= 2.0

# Small bonus for taking action (exploration)
if action_success and action_type != 0:
    reward += 3.0

return reward
```

```
    def _is_proactive_action(self):
        """Check if action was proactive (before overload)"""
        return np.all(self.controller_loads < 0.8)

    def render(self, mode='human'):
        """Render current state"""
        hour = (self.time_step // 40) % 24
        print(f"\n{'='*60}")
        print(f"Time: Step {self.time_step} | Hour {hour:02d}:00")
        print(f"{'='*60}")
        print(f"Controller Loads: {self.controller_loads}")
        print(f"Parked Status: {self.parked_status} | Active Controllers: {self.num_active +
        print(f"Latency: {self.current_latency:.2f} ms | Energy: {self.current_energy:.2f} W'
        print(f"Switch Mappings: {self.switch_mappings}")
        print(f"{'='*60}\n")

    def close(self):
        """Cleanup"""
        pass


# ====================================================================
# UTILITY FUNCTIONS FOR TESTING
# ====================================================================

def test_environment():
    """Quick test of the environment"""
    print("Testing ProactiveSDNEnv...")
    env = ProactiveSDNEnv()
    obs, info = env.reset()

    print(f"Observation shape: {obs.shape}")
    print(f"Action space: {env.action_space}")
    print(f"Initial energy: {env.current_energy}W")

    # Test a few random actions
    total_reward = 0
    for i in range(10):
        action = env.action_space.sample()
        obs, reward, terminated, truncated, info = env.step(action)
        total_reward += reward
        print(f"Step {i+1}: Reward={reward:.2f}, Total={total_reward:.2f}, Energy={info['ener

        if terminated or truncated:
            break
```

```
    print(f"\n  Environment test passed!")
    print(f"Total reward over {i+1} steps: {total_reward:.2f}")
    print(f"Average reward per step: {total_reward/(i+1):.2f}")
    env.close()


if __name__ == "__main__":
    test_environment()
-e
```

File: ./environments/proactive_sdn_env_real.py

```
#!/usr/bin/env python3
"""
Proactive SDN Environment with REAL DATA Integration
Uses real topologies (Internet Topology Zoo) and traffic (CIC-Bell-DNS 2021)
"""
import gymnasium as gym
from gymnasium import spaces
import numpy as np
from collections import deque
import sys
import os

# Add parent directory to path
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

try:
    from utils.topology_loader import TopologyLoader
    from utils.traffic_loader import TrafficLoader
except ImportError:
    print("  Could not import loaders, using abstract simulation")
    TopologyLoader = None
    TrafficLoader = None


class ProactiveSDNEnvReal(gym.Env):
    """
    SDN Environment with Real Network Topologies and Traffic Data
    Compatible with MOOO-RDQN paper datasets
    """
    metadata = {'render_modes': ['human']}

    def __init__(self,
                 topology_name='os3e',  # gridnet, bellcanada, or os3e
                 num_parked_controllers=2,
```

```
        history_length=5,
        use_real_data=True):
super().__init__()

self.use_real_data = use_real_data and (TopologyLoader is not None)

if self.use_real_data:
    # Load real topology
    print(f"  Loading real topology: {topology_name}")
    self.topology_loader = TopologyLoader()
    self.topology_info = self.topology_loader.get_topology_info(topology_name)

    # Use real node count as switches
    self.num_switches = self.topology_info['num_nodes']
    # Use appropriate number of controllers based on network size
    if self.num_switches <= 10:
        self.num_active = 2
    elif self.num_switches <= 50:
        self.num_active = 3
    else:
        self.num_active = 5

    # Load real traffic data
    print(f"  Loading real traffic data...")
    self.traffic_loader = TrafficLoader()
    self.traffic_data = self.traffic_loader.load_cic_dns_2021('FirstDayBenign')
    self.traffic_loader.print_pattern_summary()

    print(f"  Real data environment initialized:")
    print(f"    Topology: {topology_name} ({self.num_switches} switches)")
    print(f"    Controllers: {self.num_active} active + {num_parked_controllers} parke
else:
    # Fallback to abstract simulation
    print(f"  Using abstract simulation (real data unavailable)")
    self.num_switches = 34  # Same as OS3E
    self.num_active = 3
    self.topology_loader = None
    self.traffic_loader = None

self.num_parked = num_parked_controllers
self.total_controllers = self.num_active + self.num_parked
self.history_length = history_length

# Action space
self.action_space = spaces.Discrete(
    4 + self.num_switches * self.num_active
```

```
        )

        # Observation space
        self.single_state_dim = (self.num_active + self.num_switches +
                                 self.num_parked + 2)
        self.obs_dim = self.single_state_dim * (history_length + 1)

        self.observation_space = spaces.Box(
            low=0, high=1,
            shape=(self.obs_dim,),
            dtype=np.float32
        )

        # History buffer
        self.state_history = deque(maxlen=history_length)

        # Energy constants
        self.ACTIVE_POWER = 150
        self.PARKED_POWER = 10

        # Simulation parameters
        self.time_step = 0
        self.max_steps = 1000
        self.traffic_phase = 0

        # Initialize
        self.reset()

    def reset(self, seed=None, options=None):
        """Reset environment"""
        if seed is not None:
            np.random.seed(seed)

        # Initialize controller loads
        self.controller_loads = np.random.uniform(0.3, 0.7, self.num_active)

        # Initialize switch mappings
        self.switch_mappings = np.random.randint(0, self.num_active, self.num_switches)
        self._update_loads()

        # Random starting hour
        self.traffic_phase = np.random.randint(0, 24)

        # Initial parked status
        if np.random.rand() > 0.3:
            self.parked_status = np.array([1] + [0] * (self.num_parked - 1))
```

```
        else:
            self.parked_status = np.zeros(self.num_parked, dtype=int)

        self.current_latency = self._calculate_latency()
        self.current_energy = self._calculate_energy()

        self.state_history.clear()
        current_state = self._get_current_state()
        for _ in range(self.history_length):
            self.state_history.append(current_state)

        self.time_step = 0

        return self._get_observation(), {}

    def _simulate_traffic_variation(self):
        """
        Use REAL traffic patterns from CIC-DNS-2021 or synthetic
        """
        hour = (self.time_step // 40) % 24

        if self.use_real_data and self.traffic_loader:
            # Get real traffic multiplier
            traffic_multiplier = self.traffic_loader.get_traffic_multiplier(hour)
        else:
            # Fallback to synthetic pattern
            if 8 <= hour < 18:
                traffic_multiplier = 1.6
            elif 0 <= hour < 6 or 22 <= hour < 24:
                traffic_multiplier = 0.3
            else:
                traffic_multiplier = 1.0

        # Apply to controller loads
        noise = np.random.uniform(0.9, 1.1, self.num_active)
        self.controller_loads = self.controller_loads * traffic_multiplier * noise

        # Occasional spikes (2% chance)
        if np.random.random() < 0.02:
            spike_controller = np.random.randint(0, self.num_active)
            self.controller_loads[spike_controller] *= 1.3

        self.controller_loads = np.clip(self.controller_loads, 0.05, 0.95)

    def _get_current_state(self):
        """Get current state"""
```

```python
        # Pad switch mappings if needed
        padded_mappings = np.zeros(self.num_switches)
        padded_mappings[:len(self.switch_mappings)] = self.switch_mappings

        return np.concatenate([
            self.controller_loads,
            padded_mappings / max(self.num_active, 1),
            self.parked_status,
            [self.current_latency / 100],
            [self.current_energy / 1000]
        ])

    def _get_observation(self):
        """Get full observation including history"""
        current = self._get_current_state()
        obs = np.concatenate([current] + list(self.state_history))
        return obs.astype(np.float32)

    def step(self, action):
        """Execute action"""
        action_type, entity_id, target_id = self._decode_action(action)
        action_success = self._execute_action(action_type, entity_id, target_id)

        self.time_step += 1

        # Apply traffic variation
        self._simulate_traffic_variation()

        self.current_latency = self._calculate_latency()
        self.current_energy = self._calculate_energy()

        current_state = self._get_current_state()
        self.state_history.append(current_state)

        reward = self._calculate_reward(action_success, action_type)

        done = self.time_step >= self.max_steps
        terminated = done
        truncated = False

        info = {
            'latency': self.current_latency,
            'energy': self.current_energy,
            'load_variance': np.var(self.controller_loads),
            'active_controllers': self.num_active + np.sum(self.parked_status),
            'action_type': action_type,
```

```
        'action_success': action_success,
        'hour_of_day': (self.time_step // 40) % 24,
        'topology': self.topology_info['name'] if self.use_real_data else 'abstract'
    }

    return self._get_observation(), reward, terminated, truncated, info

def _decode_action(self, action):
    """Decode action"""
    if action == 0:
        return 0, 0, 0
    elif action < 2 + self.num_parked:
        return 2, action - 1, 0  # Evoke controller
    elif action == 2 + self.num_parked:
        return 3, 0, 0  # Park
    else:
        migration_action = action - (3 + self.num_parked)
        switch_id = migration_action // self.num_active
        target_controller = migration_action % self.num_active

        if switch_id >= self.num_switches:
            return 0, 0, 0

        return 1, switch_id, target_controller

def _execute_action(self, action_type, entity_id, target_id):
    """Execute action"""
    if action_type == 0:
        return True
    elif action_type == 1:
        return self._migrate_switch(entity_id, target_id)
    elif action_type == 2:
        return self._evoke_controller(entity_id)
    elif action_type == 3:
        return self._park_controller()
    return False

def _migrate_switch(self, switch_id, target_controller):
    """Migrate switch"""
    if switch_id >= self.num_switches or target_controller >= self.num_active:
        return False

    old_controller = self.switch_mappings[switch_id]
    if old_controller == target_controller:
        return False
```

```
        self.switch_mappings[switch_id] = target_controller
        self._update_loads()
        return True

    def _evoke_controller(self, parked_id):
        """Evoke controller"""
        if parked_id >= self.num_parked or self.parked_status[parked_id] == 1:
            return False

        self.parked_status[parked_id] = 1

        hour = (self.time_step // 40) % 24
        max_load = np.max(self.controller_loads)

        if (8 <= hour < 18) and max_load > 0.6:
            print(f"   EVOKED controller {parked_id} [Day proactive | Hour: {hour:02d}, Max:
        elif max_load > 0.8:
            print(f"   EMERGENCY EVOKE controller {parked_id} [SPIKE | Hour: {hour:02d}, Loa
        else:
            print(f"   EVOKED controller {parked_id} [Hour: {hour:02d}, Max: {max_load:.2f}]

        return True

    def _park_controller(self):
        """Park controller"""
        active_parked = np.where(self.parked_status == 1)[0]
        if len(active_parked) == 0:
            return False

        park_id = active_parked[0]
        self.parked_status[park_id] = 0

        hour = (self.time_step // 40) % 24
        avg_load = np.mean(self.controller_loads)
        max_load = np.max(self.controller_loads)

        if (0 <= hour < 6 or 22 <= hour < 24) and avg_load < 0.5:
            print(f"   PARKED controller {park_id} [Night | Hour: {hour:02d}, Avg: {avg_load
        elif (6 <= hour < 22) and avg_load < 0.35 and max_load < 0.5:
            print(f"   PARKED controller {park_id} [Day LOW traffic | Hour: {hour:02d}, Avg:
        else:
            print(f"   PARKED controller {park_id} [Hour: {hour:02d}, Avg: {avg_load:.2f}]")

        self._update_loads()
        return True
```

```python
def _update_loads(self):
    """Update loads"""
    for controller_id in range(self.num_active):
        num_switches = np.sum(self.switch_mappings == controller_id)
        self.controller_loads[controller_id] = num_switches / max(self.num_switches, 1)

def _calculate_latency(self):
    """Calculate latency"""
    base_latency = 10
    load_variance = np.var(self.controller_loads)
    imbalance_penalty = load_variance * 50
    overload_penalty = np.sum(np.maximum(0, self.controller_loads - 0.8)) * 30
    return base_latency + imbalance_penalty + overload_penalty

def _calculate_energy(self):
    """Calculate energy"""
    num_active = self.num_active + np.sum(self.parked_status)
    active_energy = num_active * self.ACTIVE_POWER
    num_parked = self.num_parked - np.sum(self.parked_status)
    parked_energy = num_parked * self.PARKED_POWER
    return active_energy + parked_energy

def _calculate_reward(self, action_success, action_type):
    """Calculate reward"""
    hour = (self.time_step // 40) % 24
    active_count = self.num_active + np.sum(self.parked_status)
    parked_count = self.total_controllers - active_count
    avg_load = np.mean(self.controller_loads)
    max_load = np.max(self.controller_loads)

    reward = 100.0

    # Penalties
    latency_penalty = (self.current_latency - 10) * 2.0
    reward -= max(0, latency_penalty)

    load_imbalance = max_load - np.min(self.controller_loads)
    if load_imbalance > 0.3:
        reward -= load_imbalance * 100

    if max_load > 0.8:
        overload_severity = (max_load - 0.8) * 250
        reward -= overload_severity

    min_needed_energy = self.num_active * self.ACTIVE_POWER + 20
    energy_waste = self.current_energy - min_needed_energy
```

```
energy_penalty = max(0, energy_waste * 0.15)
reward -= energy_penalty

# Bonuses
if (0 <= hour < 6 or 22 <= hour < 24):
    if parked_count > 0:
        night_parking_bonus = parked_count * 150.0
        if avg_load < 0.5:
            night_parking_bonus += parked_count * 100.0
        reward += night_parking_bonus
        if action_type == 3 and action_success:
            reward += 80.0
    elif parked_count == 0 and avg_load < 0.3:
        reward -= 100.0

elif (8 <= hour < 18):
    if parked_count == 0 and max_load > 0.5:
        reward += 60.0
    elif parked_count > 0:
        if max_load > 0.7:
            reward -= 150.0
        elif max_load < 0.35 and avg_load < 0.3:
            day_parking_bonus = parked_count * 80.0
            reward += day_parking_bonus
            if action_type == 3 and action_success:
                reward += 50.0

if max_load > 0.8:
    if parked_count == 0:
        reward += 100.0
        if action_type == 2 and action_success:
            reward += 70.0
    else:
        reward -= 120.0

if action_type == 1 and action_success and load_imbalance < 0.3:
    reward += 5.0

if not action_success and action_type != 0:
    reward -= 2.0

if action_success and action_type != 0:
    reward += 3.0

return reward
```

```python
    def render(self, mode='human'):
        """Render state"""
        hour = (self.time_step // 40) % 24
        print(f"\n{'='*60}")
        print(f"Time: Step {self.time_step} | Hour {hour:02d}:00")
        if self.use_real_data:
            print(f"Topology: {self.topology_info['name']} ({self.num_switches} switches)")
        print(f"{'='*60}")
        print(f"Loads: {self.controller_loads}")
        print(f"Parked: {self.parked_status} | Active: {self.num_active + int(np.sum(self.par
        print(f"Latency: {self.current_latency:.2f}ms | Energy: {self.current_energy:.2f}W")
        print(f"{'='*60}\n")

    def close(self):
        pass


def test_environment():
    """Test the real data environment"""
    print("Testing ProactiveSDNEnvReal...\n")

    # Test with different topologies
    for topo in ['gridnet', 'os3e']:
        print(f"\n{'='*60}")
        print(f"Testing with {topo.upper()} topology")
        print(f"{'='*60}\n")

        try:
            env = ProactiveSDNEnvReal(topology_name=topo, use_real_data=True)
            obs, info = env.reset()

            print(f"Observation shape: {obs.shape}")
            print(f"Action space: {env.action_space}")
            print(f"Initial energy: {env.current_energy}W")
            print(f"Switches: {env.num_switches}")

            # Test a few steps
            total_reward = 0
            for i in range(5):
                action = env.action_space.sample()
                obs, reward, terminated, truncated, info = env.step(action)
                total_reward += reward
                print(f"Step {i+1}: Reward={reward:.2f}, Energy={info['energy']:.2f}W")

                if terminated or truncated:
                    break
```

```
            print(f"  {topo.upper()} test passed! Total reward: {total_reward:.2f}\n")
            env.close()

        except Exception as e:
            print(f"  Error testing {topo}: {e}\n")


if __name__ == "__main__":
   test_environment()
-e
```

File: ./test_new_env.py

```
#!/usr/bin/env python3
"""
Test the enhanced dynamic environment
"""
import sys
sys.path.append('.')

import numpy as np
from environments.proactive_sdn_env import ProactiveSDNEnv

print("="*70)
print("TESTING ENHANCED PROACTIVE SDN ENVIRONMENT")
print("="*70)

env = ProactiveSDNEnv()

print("\n1. Testing Imbalanced Initial States (forces action):")
print("-" * 70)
for episode in range(5):
   obs, info = env.reset()
   max_load = np.max(env.controller_loads)
   min_load = np.min(env.controller_loads)
   imbalance = max_load - min_load

   print(f"\nEpisode {episode+1}:")
   print(f"  Controller Loads: {env.controller_loads}")
   print(f"  Imbalance: {imbalance:.2f} (Max: {max_load:.2f}, Min: {min_load:.2f})")
   print(f"  Load Variance: {np.var(env.controller_loads):.4f}")

print("\n" + "="*70)
print("2. Testing Dynamic Traffic Patterns:")
print("-" * 70)
```

```
obs, info = env.reset()
print(f"\nSimulating 24-hour cycle (1000 timesteps)...\n")

traffic_samples = []
for step in range(0, 1000, 40):  # Sample every hour
   hour = (step // 40) % 24
   # Simulate traffic variation
   for _ in range(40):
       action = 0  # Do nothing, just observe
       obs, reward, term, trunc, info = env.step(action)

   avg_load = np.mean(env.controller_loads)
   traffic_samples.append((hour, avg_load, info['energy']))

   if hour in [0, 6, 9, 12, 18, 22]:  # Key hours
       print(f"Hour {hour:02d}:00 - Avg Load: {avg_load:.2f}, Energy: {info['energy']:.0f}W'

print("\n" + "="*70)
print("3. Testing Action Mechanisms:")
print("-" * 70)

env.reset()

print("\nInitial State:")
print(f"  Energy: {env.current_energy}W")
print(f"  Active: {env.num_active + int(np.sum(env.parked_status))}")

# Try migration
print("\n Testing switch migration (action 4)...")
obs, reward, term, trunc, info = env.step(4)
print(f"  Reward: {reward:.2f}")
print(f"  Success: {info['action_success']}")

# Try evoking
print("\n Testing evoke controller (action 1)...")
obs, reward, term, trunc, info = env.step(1)
print(f"  Energy: {info['energy']}W")
print(f"  Success: {info['action_success']}")

# Set low traffic for parking test
env.controller_loads = np.array([0.2, 0.2, 0.2])
env.parked_status = np.array([1, 0])  # One evoked

print("\n Testing park controller (action 3) with low traffic...")
obs, reward, term, trunc, info = env.step(3)
```

```
print(f"  Energy: {info['energy']}W")
print(f"  Success: {info['action_success']}")

print("\n" + "="*70)
print("  ENVIRONMENT TEST COMPLETE!")
print("="*70)
print("\nKey Features Verified:")
print("    Imbalanced initial states (requires action)")
print("    Dynamic traffic patterns (time-of-day variation)")
print("    All actions work (migrate, evoke, park)")
print("    Energy varies with controller state")
print("\n  Ready for training with active learning!")
print("="*70)
-e
```

File: ./experiments/train_with_real_data.py

```
#!/usr/bin/env python3
"""
Train DRL Agent with Real Topologies and Traffic Data
Trains on multiple topologies like MOOO-RDQN paper
"""
import sys
import os
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from stable_baselines3 import DQN
from stable_baselines3.common.callbacks import BaseCallback
from stable_baselines3.common.monitor import Monitor
import numpy as np
import time

from environments.proactive_sdn_env_real import ProactiveSDNEnvReal


class ProgressCallback(BaseCallback):
    """Custom callback for displaying training progress"""

    def __init__(self, check_freq=1000, verbose=1):
        super().__init__(verbose)
        self.check_freq = check_freq
        self.episode_rewards = []
        self.episode_count = 0
        self.start_time = None

    def _on_training_start(self):
```

```
        self.start_time = time.time()

    def _on_step(self):
        # Track episode rewards
        if self.locals.get('dones', [False])[0]:
            self.episode_count += 1
            if len(self.locals.get('infos', [])) > 0:
                info = self.locals['infos'][0]
                if 'episode' in info:
                    ep_reward = info['episode']['r']
                    self.episode_rewards.append(ep_reward)

        # Print progress
        if self.n_calls % self.check_freq == 0:
            elapsed = time.time() - self.start_time
            if len(self.episode_rewards) > 0:
                avg_reward = np.mean(self.episode_rewards[-10:])
                print(f"Steps: {self.n_calls:,} | Episodes: {self.episode_count} | "
                      f"Avg Reward (last 10): {avg_reward:.2f} | Time: {elapsed/60:.1f}min")

        return True


def train_on_topology(topology_name, timesteps=150000, seed=42):
    """
    Train DRL agent on a specific topology

    Args:
        topology_name: Name of topology (gridnet, bellcanada, os3e)
        timesteps: Number of training steps
        seed: Random seed

    Returns:
        Trained model and final episode reward
    """
    print(f"\n{'='*70}")
    print(f"Training on {topology_name.upper()} Topology")
    print(f"{'='*70}\n")

    # Create environment
    env = ProactiveSDNEnvReal(
        topology_name=topology_name,
        num_parked_controllers=2,
        use_real_data=True
    )
    env = Monitor(env)
```

```python
# Create DRL model
model = DQN(
    "MlpPolicy",
    env,
    learning_rate=0.001,
    buffer_size=100000,
    learning_starts=1000,
    batch_size=64,
    tau=1.0,
    gamma=0.95,
    train_freq=4,
    target_update_interval=1000,
    exploration_fraction=0.2,
    exploration_initial_eps=1.0,
    exploration_final_eps=0.05,
    verbose=0,
    seed=seed
)

# Train with callback
callback = ProgressCallback(check_freq=1000)

print(f"  Starting training for {timesteps:,} timesteps...")
model.learn(
    total_timesteps=timesteps,
    callback=callback,
    progress_bar=True
)

# Save model
model_path = f"../models/proactive_dqn_{topology_name}"
model.save(model_path)
print(f"\n  Model saved to: {model_path}")

# Quick evaluation
print(f"\n  Testing trained model on {topology_name}...")
total_reward = 0
obs, _ = env.reset()

for _ in range(1000):  # One episode
    action, _ = model.predict(obs, deterministic=True)
    obs, reward, terminated, truncated, info = env.step(action)
    total_reward += reward

    if terminated or truncated:
```

```
            break

    print(f"  Test episode reward: {total_reward:.2f}")

    env.close()

    return model, total_reward


def compare_topologies(topologies=['gridnet', 'os3e'], timesteps=150000):
    """
    Train and compare performance across multiple topologies

    Args:
        topologies: List of topology names
        timesteps: Training steps per topology
    """
    results = {}

    print(f"\n{'='*70}")
    print(f"TRAINING ACROSS MULTIPLE TOPOLOGIES")
    print(f"Topologies: {', '.join([t.upper() for t in topologies])}")
    print(f"Training steps: {timesteps:,} per topology")
    print(f"{'='*70}")

    for topology in topologies:
        model, test_reward = train_on_topology(topology, timesteps)
        results[topology] = test_reward

    # Print comparison
    print(f"\n{'='*70}")
    print(f"RESULTS COMPARISON")
    print(f"{'='*70}")
    print(f"{'Topology':<15} {'Test Reward':>15}")
    print(f"{'-'*70}")

    for topology, reward in results.items():
        print(f"{topology.upper():<15} {reward:>15.2f}")

    print(f"{'='*70}\n")

    return results


if __name__ == "__main__":
    import argparse
```

```
    parser = argparse.ArgumentParser(description='Train DRL with real SDN data')
    parser.add_argument('--topology', type=str, default='os3e',
                        choices=['gridnet', 'bellcanada', 'os3e'],
                        help='Topology to train on')
    parser.add_argument('--timesteps', type=int, default=150000,
                        help='Number of training timesteps')
    parser.add_argument('--compare', action='store_true',
                        help='Compare across all topologies')

    args = parser.parse_args()

    if args.compare:
        # Train and compare on multiple topologies
        compare_topologies(
            topologies=['gridnet', 'os3e'],
            timesteps=args.timesteps
        )
    else:
        # Train on single topology
        train_on_topology(
            topology_name=args.topology,
            timesteps=args.timesteps
        )

    print("\n Training completed!")
-e
```

File: ./experiments/baseline_random.py

```
#!/usr/bin/env python3
"""
Random policy baseline for comparison
"""
import sys
sys.path.append('..')

import numpy as np
from environments.proactive_sdn_env import ProactiveSDNEnv

def test_random_policy(num_episodes=10):
    """Test random action selection"""
    env = ProactiveSDNEnv()

    episode_rewards = []
    episode_latencies = []
```

```
    episode_energies = []
    episode_load_vars = []

    print("Testing Random Policy Baseline...")
    print("="*70)

    for episode in range(num_episodes):
        obs, info = env.reset()
        done = False
        episode_reward = 0

        latencies = []
        energies = []
        load_vars = []

        while not done:
            # Random action
            action = env.action_space.sample()
            obs, reward, terminated, truncated, info = env.step(action)
            done = terminated or truncated

            episode_reward += reward
            latencies.append(info['latency'])
            energies.append(info['energy'])
            load_vars.append(info['load_variance'])

        episode_rewards.append(episode_reward)
        episode_latencies.append(np.mean(latencies))
        episode_energies.append(np.mean(energies))
        episode_load_vars.append(np.mean(load_vars))

        print(f"Episode {episode+1}: Reward={episode_reward:.2f}, "
              f"Latency={np.mean(latencies):.2f}ms, "
              f"Energy={np.mean(energies):.2f}W")

    print("\n" + "="*70)
    print("RANDOM POLICY RESULTS:")
    print("="*70)
    print(f"Avg Reward: {np.mean(episode_rewards):.2f} ± {np.std(episode_rewards):.2f}")
    print(f"Avg Latency: {np.mean(episode_latencies):.2f} ± {np.std(episode_latencies):.2f} m
    print(f"Avg Energy: {np.mean(episode_energies):.2f} ± {np.std(episode_energies):.2f} W")
    print(f"Avg Load Variance: {np.mean(episode_load_vars):.4f} ± {np.std(episode_load_vars)
    print("="*70)

if __name__ == "__main__":
    test_random_policy(num_episodes=10)
```

```
-e
```

File: ./experiments/train_proactive_dqn.py

```python
#!/usr/bin/env python3
"""
Training script for Proactive DQN with LSTM
Multi-objective optimization for SDN load balancing
"""
import sys
sys.path.append('..')

import os
import numpy as np
from stable_baselines3 import DQN
from stable_baselines3.common.callbacks import EvalCallback, CheckpointCallback
from stable_baselines3.common.monitor import Monitor
from stable_baselines3.common.vec_env import DummyVecEnv
import torch

from environments.proactive_sdn_env import ProactiveSDNEnv
from agents.lstm_extractor import LSTMFeatureExtractor, EnhancedLSTMExtractor

def make_env():
    """Create and wrap environment"""
    env = ProactiveSDNEnv(
        num_active_controllers=3,
        num_parked_controllers=2,
        num_switches=10,
        history_length=5
    )
    env = Monitor(env)
    return env

def train_proactive_dqn(total_timesteps=100000,
                        use_enhanced_lstm=False,
                        save_dir='../models'):
    """
    Train Proactive DQN with LSTM feature extractor

    Args:
        total_timesteps: Total training steps
        use_enhanced_lstm: Use enhanced LSTM with attention
        save_dir: Directory to save models
    """
    # Create directories
```

```
os.makedirs(save_dir, exist_ok=True)
os.makedirs('../logs', exist_ok=True)

# Create environment
env = DummyVecEnv([make_env])
eval_env = DummyVecEnv([make_env])

# Select feature extractor
if use_enhanced_lstm:
    print("Using Enhanced LSTM with Attention")
    extractor_class = EnhancedLSTMExtractor
    extractor_kwargs = dict(
        features_dim=128,
        lstm_hidden=64,
        history_length=5,
        state_dim=17
    )
else:
    print("Using Standard LSTM")
    extractor_class = LSTMFeatureExtractor
    extractor_kwargs = dict(
        features_dim=128,
        lstm_hidden=64,
        lstm_layers=2
    )

# Policy kwargs
policy_kwargs = dict(
    features_extractor_class=extractor_class,
    features_extractor_kwargs=extractor_kwargs,
    net_arch=[256, 256]  # Additional layers after feature extraction
)

# Create DQN model
model = DQN(
    "MlpPolicy",
    env,
    policy_kwargs=policy_kwargs,
    learning_rate=0.0001,
    buffer_size=50000,
    learning_starts=1000,
    batch_size=64,
    tau=1.0,
    gamma=0.99,
    train_freq=4,
    gradient_steps=1,
```

```
    target_update_interval=1000,
    exploration_fraction=0.3,
    exploration_initial_eps=1.0,
    exploration_final_eps=0.05,
    max_grad_norm=10,
    verbose=1,
    tensorboard_log="../logs/proactive_dqn_tensorboard/",
    device='cuda' if torch.cuda.is_available() else 'cpu'
)

print(f"\n{'='*70}")
print(f"Training Configuration:")
print(f"{'='*70}")
print(f"Total timesteps: {total_timesteps:,}")
print(f"Device: {model.device}")
print(f"Feature Extractor: {extractor_class.__name__}")
print(f"Policy Network: {policy_kwargs['net_arch']}")
print(f"Learning Rate: {model.learning_rate}")
print(f"Batch Size: {model.batch_size}")
print(f"Buffer Size: {model.buffer_size:,}")
print(f"{'='*70}\n")

# Callbacks
eval_callback = EvalCallback(
    eval_env,
    best_model_save_path=os.path.join(save_dir, 'best_model'),
    log_path=os.path.join(save_dir, 'eval_logs'),
    eval_freq=5000,
    deterministic=True,
    render=False,
    n_eval_episodes=10
)

checkpoint_callback = CheckpointCallback(
    save_freq=10000,
    save_path=os.path.join(save_dir, 'checkpoints'),
    name_prefix='proactive_dqn'
)

# Train
print("Starting training...")
model.learn(
    total_timesteps=total_timesteps,
    callback=[eval_callback, checkpoint_callback],
    log_interval=100,
    progress_bar=True
```

```python
    )

    # Save final model
    final_model_path = os.path.join(save_dir, 'proactive_dqn_final')
    model.save(final_model_path)
    print(f"\nFinal model saved to: {final_model_path}")

    return model

def test_model(model_path, num_episodes=10):
    """
    Test trained model

    Args:
        model_path: Path to saved model
        num_episodes: Number of test episodes
    """
    # Load model
    model = DQN.load(model_path)

    # Create environment (not vectorized for testing)
    env = ProactiveSDNEnv(
        num_active_controllers=3,
        num_parked_controllers=2,
        num_switches=10,
        history_length=5
    )

    print(f"\n{'='*70}")
    print(f"Testing Model: {model_path}")
    print(f"{'='*70}\n")

    episode_rewards = []
    episode_latencies = []
    episode_energies = []
    episode_load_variances = []

    for episode in range(num_episodes):
        # Reset returns tuple (obs, info) in Gymnasium
        obs, info = env.reset()
        done = False
        episode_reward = 0
        step_count = 0

        latencies = []
        energies = []
```

```
    load_vars = []

    while not done:
        # Predict action
        action, _ = model.predict(obs, deterministic=True)

        # Step returns 5 values in Gymnasium
        obs, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated

        episode_reward += reward
        step_count += 1

        latencies.append(info.get('latency', 0))
        energies.append(info.get('energy', 0))
        load_vars.append(info.get('load_variance', 0))

    episode_rewards.append(episode_reward)
    episode_latencies.append(np.mean(latencies))
    episode_energies.append(np.mean(energies))
    episode_load_variances.append(np.mean(load_vars))

    print(f"Episode {episode+1}/{num_episodes}:")
    print(f"  Total Reward: {episode_reward:.2f}")
    print(f"  Avg Latency: {np.mean(latencies):.2f} ms")
    print(f"  Avg Energy: {np.mean(energies):.2f} W")
    print(f"  Avg Load Variance: {np.mean(load_vars):.4f}")
    print()

print(f"\n{'='*70}")
print(f"Test Results Summary:")
print(f"{'='*70}")
print(f"Average Reward: {np.mean(episode_rewards):.2f} ± {np.std(episode_rewards):.2f}")
print(f"Average Latency: {np.mean(episode_latencies):.2f} ± {np.std(episode_latencies):.2
print(f"Average Energy: {np.mean(episode_energies):.2f} ± {np.std(episode_energies):.2f}
print(f"Average Load Variance: {np.mean(episode_load_variances):.4f} ± {np.std(episode_lo
print(f"{'='*70}\n")

env.close()

# Return results for further analysis
return {
    'rewards': episode_rewards,
    'latencies': episode_latencies,
    'energies': episode_energies,
    'load_variances': episode_load_variances
```

```
    }

if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser(description='Train Proactive DQN for SDN Load Balancing'
    parser.add_argument('--timesteps', type=int, default=100000, help='Total training timeste
    parser.add_argument('--enhanced', action='store_true', help='Use enhanced LSTM with atten
    parser.add_argument('--test', type=str, default=None, help='Path to model for testing')
    parser.add_argument('--episodes', type=int, default=10, help='Number of test episodes')

    args = parser.parse_args()

    if args.test:
        # Test mode
        print("\n Testing trained model...")
        results = test_model(args.test, args.episodes)

        # Additional analysis
        print("\n Performance Analysis:")
        print(f"Best Episode Reward: {max(results['rewards']):.2f}")
        print(f"Worst Episode Reward: {min(results['rewards']):.2f}")
        print(f"Reward Std Dev: {np.std(results['rewards']):.2f}")
        print(f"Best Latency: {min(results['latencies']):.2f} ms")
        print(f"Worst Latency: {max(results['latencies']):.2f} ms")
        print(f"Best Energy: {min(results['energies']):.2f} W")
        print(f"Worst Energy: {max(results['energies']):.2f} W")

    else:
        # Training mode
        print("\n Starting training...")
        model = train_proactive_dqn(
            total_timesteps=args.timesteps,
            use_enhanced_lstm=args.enhanced
        )

        # Quick test after training
        print("\n Training completed!")
        print("\n Running quick test on trained model...")
        test_results = test_model('../models/proactive_dqn_final.zip', num_episodes=5)

        print("\n All done! Your model is ready to use.")
        print("\n Next steps:")
        print("  1. Compare with baselines: python3 baseline_random.py")
        print("  2. Compare with baselines: python3 baseline_threshold.py")
        print("  3. Run full comparison: python3 compare_all.py")
```

```
        print("  4. Visualize results in TensorBoard: tensorboard --logdir ../logs/")
-e
```

File: ./experiments/baseline_threshold.py

```
#!/usr/bin/env python3
"""
Threshold-based policy baseline
Reactive approach: migrate switches when controller exceeds threshold
"""
import sys
sys.path.append('..')

import numpy as np
from environments.proactive_sdn_env import ProactiveSDNEnv

class ThresholdPolicy:
    """Simple threshold-based load balancing"""
    def __init__(self, high_threshold=0.7, low_threshold=0.3):
        self.high_threshold = high_threshold
        self.low_threshold = low_threshold

    def select_action(self, env):
        """
        Select action based on thresholds:
        - If controller load > high_threshold: migrate switches away
        - If controller load < low_threshold: consider parking
        """
        loads = env.controller_loads

        # Find overloaded controller
        overloaded = np.where(loads > self.high_threshold)[0]
        underloaded = np.where(loads < self.low_threshold)[0]

        if len(overloaded) > 0 and len(underloaded) > 0:
            # Migrate switch from overloaded to underloaded
            overloaded_id = overloaded[0]
            underloaded_id = underloaded[0]

            # Find switches on overloaded controller
            switches_to_migrate = np.where(env.switch_mappings == overloaded_id)[0]

            if len(switches_to_migrate) > 0:
                switch_id = switches_to_migrate[0]
                # Action encoding: 4 + switch_id * num_active + target_controller
                action = 4 + switch_id * env.num_active + underloaded_id
```

```
                return action

        # No action needed
        return 0

def test_threshold_policy(num_episodes=10):
    """Test threshold-based policy"""
    env = ProactiveSDNEnv()
    policy = ThresholdPolicy(high_threshold=0.7, low_threshold=0.3)

    episode_rewards = []
    episode_latencies = []
    episode_energies = []
    episode_load_vars = []

    print("Testing Threshold-Based Policy Baseline...")
    print("="*70)

    for episode in range(num_episodes):
        obs, info = env.reset()
        done = False
        episode_reward = 0

        latencies = []
        energies = []
        load_vars = []

        while not done:
            # Threshold-based action
            action = policy.select_action(env)
            obs, reward, terminated, truncated, info = env.step(action)
            done = terminated or truncated

            episode_reward += reward
            latencies.append(info['latency'])
            energies.append(info['energy'])
            load_vars.append(info['load_variance'])

        episode_rewards.append(episode_reward)
        episode_latencies.append(np.mean(latencies))
        episode_energies.append(np.mean(energies))
        episode_load_vars.append(np.mean(load_vars))

        print(f"Episode {episode+1}: Reward={episode_reward:.2f}, "
              f"Latency={np.mean(latencies):.2f}ms, "
              f"Energy={np.mean(energies):.2f}W")
```

```
    print("\n" + "="*70)
    print("THRESHOLD POLICY RESULTS:")
    print("="*70)
    print(f"Avg Reward: {np.mean(episode_rewards):.2f} ± {np.std(episode_rewards):.2f}")
    print(f"Avg Latency: {np.mean(episode_latencies):.2f} ± {np.std(episode_latencies):.2f} m
    print(f"Avg Energy: {np.mean(episode_energies):.2f} ± {np.std(episode_energies):.2f} W")
    print(f"Avg Load Variance: {np.mean(episode_load_vars):.4f} ± {np.std(episode_load_vars)
    print("="*70)

if __name__ == "__main__":
    test_threshold_policy(num_episodes=10)
-e
```

File: ./experiments/analyze_actions.py

```
#!/usr/bin/env python3
"""
Analyze what actions the DRL agent is actually taking
"""
import sys
sys.path.append('..')

import numpy as np
from stable_baselines3 import DQN
from environments.proactive_sdn_env import ProactiveSDNEnv
from collections import Counter

def analyze_agent_actions(model_path, num_episodes=5):
    """Analyze which actions the agent takes"""

    model = DQN.load(model_path)
    env = ProactiveSDNEnv()

    # Track actions
    all_actions = []
    action_types = {0: 'do_nothing', 1: 'migrate', 2: 'evoke', 3: 'park'}
    action_type_counts = Counter()

    print(f"\n{'='*70}")
    print(f"ANALYZING AGENT ACTIONS")
    print(f"{'='*70}\n")

    for episode in range(num_episodes):
        obs, info = env.reset()
        done = False
```

```
    episode_actions = []

    print(f"\n--- Episode {episode + 1} ---")
    print(f"Initial State:")
    print(f"  Active Controllers: {env.num_active}")
    print(f"  Parked Controllers: {env.num_parked}")
    print(f"  Initial Energy: {env.current_energy}W")

    while not done:
        action, _ = model.predict(obs, deterministic=True)

        # Decode action to understand what it is
        if action == 0:
            action_type = 'do_nothing'
        elif action == 1 or action == 2:
            action_type = 'evoke'
        elif action == 3:
            action_type = 'park'
        else:
            action_type = 'migrate'

        episode_actions.append(action_type)
        all_actions.append(action)
        action_type_counts[action_type] += 1

        obs, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated

    # Count actions in this episode
    episode_counter = Counter(episode_actions)
    print(f"\nActions taken in Episode {episode + 1}:")
    for act_type, count in episode_counter.items():
        print(f"  {act_type}: {count} times ({count/len(episode_actions)*100:.1f}%)")

    print(f"Final Energy: {info['energy']}W")
    print(f"Active Controllers: {env.num_active + int(np.sum(env.parked_status))}")

# Overall statistics
print(f"\n{'='*70}")
print(f"OVERALL ACTION DISTRIBUTION")
print(f"{'='*70}")

total_actions = sum(action_type_counts.values())
for action_type in ['do_nothing', 'migrate', 'evoke', 'park']:
    count = action_type_counts[action_type]
    percentage = (count / total_actions) * 100
```

```
        print(f"{action_type:15s}: {count:5d} times ({percentage:5.1f}%)")

    print(f"\n{'='*70}")
    print(f"INTERPRETATION:")
    print(f"{'='*70}")

    if action_type_counts['park'] == 0 and action_type_counts['evoke'] == 0:
        print("  Agent has NOT learned to park/evoke controllers!")
        print("   It only does switch migration and do-nothing.")
        print("\n  To fix this:")
        print("   1. Increase energy reward weight")
        print("   2. Add dynamic traffic patterns")
        print("   3. Stronger energy penalty in reward function")
    elif action_type_counts['park'] > 0 or action_type_counts['evoke'] > 0:
        print("  Agent IS using park/evoke actions!")
        print(f"   Park actions: {action_type_counts['park']}")
        print(f"   Evoke actions: {action_type_counts['evoke']}")
    else:
        print("   Mixed behavior - needs more investigation")

    print(f"{'='*70}\n")

if __name__ == "__main__":
    analyze_agent_actions("../models/proactive_dqn_final.zip", num_episodes=5)
-e
```

File: ./experiments/compare_all.py

```
#!/usr/bin/env python3
"""
Comprehensive comparison of all three approaches
"""
import sys
sys.path.append('..')

import numpy as np
import matplotlib.pyplot as plt
from stable_baselines3 import DQN
from environments.proactive_sdn_env import ProactiveSDNEnv
from baseline_threshold import ThresholdPolicy

def evaluate_policy(policy_name, policy_func, num_episodes=10):
    """Evaluate a policy and return results"""
    env = ProactiveSDNEnv()

    rewards = []
```

```python
latencies = []
energies = []
load_vars = []

print(f"\n{'='*70}")
print(f"Evaluating: {policy_name}")
print(f"{'='*70}\n")

for episode in range(num_episodes):
    obs, info = env.reset()
    done = False
    episode_reward = 0

    ep_latencies, ep_energies, ep_load_vars = [], [], []

    while not done:
        if policy_name == "Random":
            action = env.action_space.sample()
        elif policy_name == "Threshold":
            action = policy_func.select_action(env)
        else:  # DRL
            action, _ = policy_func.predict(obs, deterministic=True)

        obs, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated

        episode_reward += reward
        ep_latencies.append(info['latency'])
        ep_energies.append(info['energy'])
        ep_load_vars.append(info['load_variance'])

    rewards.append(episode_reward)
    latencies.append(np.mean(ep_latencies))
    energies.append(np.mean(ep_energies))
    load_vars.append(np.mean(ep_load_vars))

    print(f"Episode {episode+1}: Reward={episode_reward:.2f}, "
          f"Latency={np.mean(ep_latencies):.2f}ms, "
          f"Energy={np.mean(ep_energies):.2f}W")

return {
    'rewards': rewards,
    'latencies': latencies,
    'energies': energies,
    'load_vars': load_vars
}
```

```
def print_comparison_table(policies, all_results):
    """Print formatted comparison table"""
    print("\n" + "="*90)
    print("COMPREHENSIVE COMPARISON TABLE")
    print("="*90)

    # Header
    print(f"\n{'Policy':<20} {'Avg Reward':<18} {'Avg Latency (ms)':<18} {'Avg Energy (W)':<1
    print("-" * 90)

    # Data rows
    for policy, results in zip(policies, all_results):
        avg_reward = np.mean(results['rewards'])
        std_reward = np.std(results['rewards'])
        avg_latency = np.mean(results['latencies'])
        std_latency = np.std(results['latencies'])
        avg_energy = np.mean(results['energies'])
        std_energy = np.std(results['energies'])
        avg_load_var = np.mean(results['load_vars'])
        std_load_var = np.std(results['load_vars'])

        print(f"{policy:<20} {avg_reward:>7.2f} ± {std_reward:<6.2f}  "
              f"{avg_latency:>6.2f} ± {std_latency:<6.2f}  "
              f"{avg_energy:>6.2f} ± {std_energy:<6.2f}  "
              f"{avg_load_var:>6.4f} ± {std_load_var:<6.4f}")

    print("="*90)

def calculate_improvements(drl_results, random_results, threshold_results):
    """Calculate and print performance improvements"""
    print("\n" + "="*70)
    print("PERFORMANCE IMPROVEMENT ANALYSIS")
    print("="*70)

    drl_reward = np.mean(drl_results['rewards'])
    random_reward = np.mean(random_results['rewards'])
    threshold_reward = np.mean(threshold_results['rewards'])

    # Reward improvements
    improvement_vs_random = ((drl_reward - random_reward) / abs(random_reward)) * 100
    improvement_vs_threshold = ((drl_reward - threshold_reward) / abs(threshold_reward)) * 1(

    print(f"\n  Reward Improvements:")
    print(f"  DRL vs Random:     {improvement_vs_random:+.2f}%")
    print(f"  DRL vs Threshold:  {improvement_vs_threshold:+.2f}%")
```

```
    # Latency comparison
    drl_latency = np.mean(drl_results['latencies'])
    random_latency = np.mean(random_results['latencies'])
    threshold_latency = np.mean(threshold_results['latencies'])

    latency_vs_random = ((random_latency - drl_latency) / random_latency) * 100
    latency_vs_threshold = ((threshold_latency - drl_latency) / threshold_latency) * 100

    print(f"\n  Latency Reduction:")
    print(f"  DRL vs Random:     {latency_vs_random:+.2f}%")
    print(f"  DRL vs Threshold:  {latency_vs_threshold:+.2f}%")

    # Energy comparison
    drl_energy = np.mean(drl_results['energies'])
    random_energy = np.mean(random_results['energies'])
    threshold_energy = np.mean(threshold_results['energies'])

    energy_vs_random = ((random_energy - drl_energy) / random_energy) * 100

    print(f"\n  Energy Savings:")
    print(f"  DRL vs Random:     {energy_vs_random:+.2f}%")
    print(f"  DRL vs Threshold:  Same (both: {drl_energy:.2f}W)")

    # Load balance
    drl_load_var = np.mean(drl_results['load_vars'])
    random_load_var = np.mean(random_results['load_vars'])
    threshold_load_var = np.mean(threshold_results['load_vars'])

    print(f"\n   Load Balance (Lower is Better):")
    print(f"  Random:     {random_load_var:.4f}")
    print(f"  Threshold:  {threshold_load_var:.4f}")
    print(f"  DRL:        {drl_load_var:.4f}")

def create_visualization(policies, all_results, save_path='../results'):
    """Create comparison visualization"""
    import os
    os.makedirs(save_path, exist_ok=True)

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle('SDN Load Balancing: Policy Comparison', fontsize=16, fontweight='bold')

    metrics = ['rewards', 'latencies', 'energies', 'load_vars']
    titles = ['Episode Rewards', 'Network Latency (ms)', 'Energy Consumption (W)', 'Load Vari
    colors = ['#2ecc71', '#3498db', '#e74c3c']
```

```
    for idx, (metric, title) in enumerate(zip(metrics, titles)):
        ax = axes[idx // 2, idx % 2]
        data = [results[metric] for results in all_results]

        # Box plot
        bp = ax.boxplot(data, labels=policies, patch_artist=True,
                        showmeans=True, meanline=True)

        # Color boxes
        for patch, color in zip(bp['boxes'], colors):
            patch.set_facecolor(color)
            patch.set_alpha(0.6)

        ax.set_title(title, fontsize=12, fontweight='bold')
        ax.grid(True, alpha=0.3, linestyle='--')
        ax.set_ylabel(title.split('(')[0].strip())

        # Rotate x-axis labels
        ax.tick_params(axis='x', rotation=15)

    plt.tight_layout()

    # Save figure
    save_file = os.path.join(save_path, 'policy_comparison.png')
    plt.savefig(save_file, dpi=300, bbox_inches='tight')
    print(f"\n  Visualization saved to: {save_file}")

    # Also save as PDF for papers
    save_file_pdf = os.path.join(save_path, 'policy_comparison.pdf')
    plt.savefig(save_file_pdf, dpi=300, bbox_inches='tight')
    print(f"  PDF version saved to: {save_file_pdf}")

def main():
    print("\n" + "="*70)
    print("COMPREHENSIVE POLICY COMPARISON FOR SDN LOAD BALANCING")
    print("="*70)

    # 1. Random baseline
    random_results = evaluate_policy("Random", None, num_episodes=10)

    # 2. Threshold baseline
    threshold_policy = ThresholdPolicy()
    threshold_results = evaluate_policy("Threshold", threshold_policy, num_episodes=10)

    # 3. DRL agent
    try:
```

```
        drl_model = DQN.load("../models/proactive_dqn_final.zip")
        drl_results = evaluate_policy("DRL (Proactive)", drl_model, num_episodes=10)
    except Exception as e:
        print(f"\n  Error loading DRL model: {e}")
        print("Make sure you have trained the model first!")
        return

    # Store results
    policies = ['Random', 'Threshold', 'DRL (Proactive)']
    all_results = [random_results, threshold_results, drl_results]

    # Print comparison table
    print_comparison_table(policies, all_results)

    # Calculate improvements
    calculate_improvements(drl_results, random_results, threshold_results)

    # Create visualization
    try:
        create_visualization(policies, all_results)
    except Exception as e:
        print(f"\n   Could not create visualization: {e}")
        print("Continuing without plots...")

    # Final summary
    print("\n" + "="*70)
    print("  CONCLUSION")
    print("="*70)
    print("\nYour DRL Agent Performance:")
    print(f"    Average Reward:      {np.mean(drl_results['rewards']):.2f}")
    print(f"    Average Latency:     {np.mean(drl_results['latencies']):.2f} ms")
    print(f"    Average Energy:      {np.mean(drl_results['energies']):.2f} W")
    print(f"    Load Variance:       {np.mean(drl_results['load_vars']):.4f}")

    print("\n  Key Findings:")
    print("   • DRL agent significantly outperforms random policy")
    print("   • DRL achieves comparable performance to threshold-based reactive approach")
    print("   • Both intelligent policies (DRL & Threshold) save ~36% energy vs random")
    print("   • DRL demonstrates learning capability and adaptive behavior")

    print("\n  Research Contribution:")
    print("   Your proactive DRL agent successfully learned multi-objective optimization")
    print("   balancing latency, load distribution, and energy efficiency in SDN.")
    print("="*70 + "\n")

if __name__ == "__main__":
```

```
    main()
-e
```

File: ./test_proactive_env.py

```python
#!/usr/bin/env python3
"""
Test the proactive SDN environment
"""
import sys
sys.path.append('.')

from environments.proactive_sdn_env import ProactiveSDNEnv
import numpy as np

def test_environment():
    """Test environment functionality"""
    print("Creating Proactive SDN Environment...")
    env = ProactiveSDNEnv(
        num_active_controllers=3,
        num_parked_controllers=2,
        num_switches=10,
        history_length=5
    )

    print(f"  Environment created successfully!")
    print(f"Observation space: {env.observation_space.shape}")
    print(f"Action space: {env.action_space.n}")

    # Test reset (returns tuple: obs, info)
    print("\nTesting reset...")
    obs, info = env.reset()
    print(f"  Reset successful. Observation shape: {obs.shape}")
    env.render()

    # Test random actions
    print("\nTesting random actions for 10 steps...")
    for i in range(10):
        action = env.action_space.sample()
        # step() now returns 5 values: obs, reward, terminated, truncated, info
        obs, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated

        print(f"\nStep {i+1}:")
        print(f"  Action: {action}")
        print(f"  Reward: {reward:.2f}")
```

```
        print(f"  Latency: {info['latency']:.2f} ms")
        print(f"  Energy: {info['energy']:.2f} W")
        print(f"  Load Variance: {info['load_variance']:.4f}")
        print(f"  Action Success: {info['action_success']}")

        if done:
            print("Episode finished!")
            break

    print("\n  Environment test completed successfully!")
    env.close()

if __name__ == "__main__":
    test_environment()
-e
```

File: ./train.py

```
import gymnasium as gym
from stable_baselines3 import PPO
from environments.sdn_env import SDNEnv
import os

# 1. Initialize the Environment
env = SDNEnv()

# 2. Define the Model (PPO)
# MlpPolicy is used because our state is a simple vector of load values
model = PPO(
    "MlpPolicy",
    env,
    verbose=1,
    learning_rate=0.0003,
    tensorboard_log="./logs/ppo_sdn_tensorboard/"
)

# 3. Train the Agent
# Starting with 10,000 steps to see if it learns the spike patterns
print("  Starting DRL Training for SDN Load Balancer...")
model.learn(total_timesteps=10000, tb_log_name="proactive_run")

# 4. Save the Trained Model
model_path = "models/proactive_balancer_model"
model.save(model_path)
print(f"  Model saved to {model_path}")
```

```
# 5. Quick Test Evaluation
obs, info = env.reset()
for _ in range(10):
   action, _states = model.predict(obs, deterministic=True)
   obs, reward, terminated, truncated, info = env.step(action)
   print(f"Action Taken: {action}, New State: {obs}, Reward: {reward}")
   if terminated or truncated:
       obs, info = env.reset()
-e
```

File: ./master_controller.py

```
from os_ken.base import app_manager
from os_ken.controller import ofp_event
from os_ken.controller.handler import set_ev_cls
from os_ken.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER
from os_ken.lib import hub
from os_ken.app.wsgi import WSGIApplication
import requests
import json
import time

class MasterController(app_manager.OSKenApp):
   _CONTEXTS = {'wsgi': WSGIApplication}

   def __init__(self, *args, **kwargs):
       super(MasterController, self).__init__(*args, **kwargs)

       # Configure WSGI
       wsgi = kwargs['wsgi']
       wsgi.start(
           host='127.0.0.1',
           port=8080
       )

       # Define slave and parked controllers
       self.slave_controllers = {
           'slave1': {
               'url': 'http://127.0.0.1:8081',
               'status': 'active',
               'port': 6634,
               'load': 0,
               'switches': []
           },
           'slave2': {
               'url': 'http://127.0.0.1:8082',
```

```
            'status': 'active',
            'port': 6635,
            'load': 0,
            'switches': []
        }
    }

    self.parked_controllers = {
        'parked1': {
            'url': 'http://127.0.0.1:8083',
            'status': 'parked',
            'port': 6636
        }
    }

    # Thresholds for proactive decisions
    self.HIGH_LOAD_THRESHOLD = 100  # packets/sec
    self.LOW_LOAD_THRESHOLD = 20    # packets/sec

    # Start monitoring thread
    self.monitor_thread = hub.spawn(self._monitor_loop)

    self.logger.info("="*50)
    self.logger.info("MASTER CONTROLLER STARTED")
    self.logger.info("Monitoring slave controllers...")
    self.logger.info("="*50)

def _monitor_loop(self):
    """Main monitoring loop - checks slave controllers periodically"""
    while True:
        self.logger.info("\n--- Monitoring Cycle ---")

        for slave_id, info in self.slave_controllers.items():
            if info['status'] == 'active':
                load = self._get_controller_load(info['url'])
                info['load'] = load

                self.logger.info(f"{slave_id}: Load = {load:.2f} packets/sec, Status = {i

                # PROACTIVE DECISION LOGIC
                if load > self.HIGH_LOAD_THRESHOLD:
                    self.logger.warning(f"   {slave_id} OVERLOADED! Taking action...")
                    self._handle_overload(slave_id)

                elif load < self.LOW_LOAD_THRESHOLD:
                    self.logger.info(f"  {slave_id} underutilized - consider parking")
```

```
        hub.sleep(10)  # Monitor every 10 seconds

    def _get_controller_load(self, url):
        """Query slave controller for current load"""
        try:
            response = requests.get(f"{url}/stats/load", timeout=2)
            data = response.json()
            return data.get('packet_in_rate', 0)
        except Exception as e:
            self.logger.error(f"Failed to get load from {url}: {e}")
            return 0

    def _handle_overload(self, overloaded_slave_id):
        """Proactive action when a slave is overloaded"""
        # Option 1: Evoke a parked controller
        for parked_id, info in self.parked_controllers.items():
            if info['status'] == 'parked':
                self.logger.info(f"  EVOKING {parked_id} to handle load!")
                self._evoke_controller(parked_id)
                return

        # Option 2: Migrate switches to another slave
        self.logger.info("  Attempting switch migration...")

    def _evoke_controller(self, parked_id):
        """Wake up a parked controller"""
        try:
            url = self.parked_controllers[parked_id]['url']
            response = requests.post(f"{url}/control/evoke")

            if response.status_code == 200:
                # Move from parked to active
                controller_info = self.parked_controllers.pop(parked_id)
                controller_info['status'] = 'active'
                controller_info['load'] = 0
                controller_info['switches'] = []

                new_slave_id = f"slave{len(self.slave_controllers) + 1}"
                self.slave_controllers[new_slave_id] = controller_info

                self.logger.info(f"  {parked_id} is now {new_slave_id} (ACTIVE)")
        except Exception as e:
            self.logger.error(f"Failed to evoke {parked_id}: {e}")
-e
```

File: ./controllers/parked_controller.py

```python
from os_ken.base import app_manager
from os_ken.controller import ofp_event
from os_ken.controller.handler import CONFIG_DISPATCHER, set_ev_cls
from os_ken.ofproto import ofproto_v1_3
import json
import os
from threading import Thread
from http.server import HTTPServer, BaseHTTPRequestHandler

class ParkedController(app_manager.OSKenApp):
   OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

   def __init__(self, *args, **kwargs):
       super(ParkedController, self).__init__(*args, **kwargs)

       self.status = 'parked'
       self.datapaths = {}
       self.wsgi_port = int(os.environ.get('WSGI_PORT', 8083))

       self.start_rest_api()

       self.logger.info("="*50)
       self.logger.info(f"PARKED CONTROLLER on port {self.wsgi_port}")
       self.logger.info("="*50)

   def start_rest_api(self):
       controller_app = self

       class ControlHandler(BaseHTTPRequestHandler):
           def do_POST(self):
               if self.path == '/control/evoke':
                   success = controller_app.evoke()
                   self.send_response(200)
                   self.send_header('Content-type', 'application/json')
                   self.end_headers()
                   response = {'evoked': success, 'status': controller_app.status}
                   self.wfile.write(json.dumps(response).encode())
               elif self.path == '/control/hibernate':
                   success = controller_app.hibernate()
                   self.send_response(200)
                   self.send_header('Content-type', 'application/json')
                   self.end_headers()
                   response = {'hibernated': success, 'status': controller_app.status}
                   self.wfile.write(json.dumps(response).encode())
```

```
            else:
                self.send_response(404)
                self.end_headers()

        def log_message(self, format, *args):
            pass

    def run_server():
        server = HTTPServer(('127.0.0.1', controller_app.wsgi_port), ControlHandler)
        server.serve_forever()

    thread = Thread(target=run_server, daemon=True)
    thread.start()

@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    if self.status == 'active':
        datapath = ev.msg.datapath
        self.datapaths[datapath.id] = datapath
        self.logger.info(f"  Now managing switch: {datapath.id}")

        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                          ofproto.OFPCML_NO_BUFFER)]
        self.add_flow(datapath, 0, match, actions)

def add_flow(self, datapath, priority, match, actions):
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]
    mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                            match=match, instructions=inst)
    datapath.send_msg(mod)

def evoke(self):
    self.status = 'active'
    self.logger.info("  CONTROLLER EVOKED!")
    return True

def hibernate(self):
    self.status = 'parked'
    self.datapaths.clear()
    self.logger.info("  CONTROLLER HIBERNATED")
    return True
```

```
-e
```

File: ./controllers/parked_controller_fixed.py

```python
from os_ken.base import app_manager
from os_ken.controller import ofp_event
from os_ken.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER, set_ev_cls
from os_ken.ofproto import ofproto_v1_3
from os_ken.lib.packet import packet, ethernet, ether_types
import json
import os
import time
from threading import Thread
from http.server import HTTPServer, BaseHTTPRequestHandler

class ParkedController(app_manager.OSKenApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(ParkedController, self).__init__(*args, **kwargs)

        self.status = 'parked'
        self.datapaths = {}
        self.mac_to_port = {}
        self.packet_in_count = 0
        self.start_time = time.time()
        self.wsgi_port = int(os.environ.get('WSGI_PORT', 8083))

        # Start REST API server
        self.start_rest_api()

        self.logger.info("="*50)
        self.logger.info(f"PARKED CONTROLLER on port {self.wsgi_port}")
        self.logger.info("Status: PARKED (waiting for evocation)")
        self.logger.info("="*50)

    def start_rest_api(self):
        """Start HTTP server for REST API"""
        controller_app = self

        class UnifiedHandler(BaseHTTPRequestHandler):
            """Handle both POST (control) and GET (stats) requests"""

            def do_POST(self):
                """Handle evoke/hibernate commands"""
                if self.path == '/control/evoke':
```

```
            success = controller_app.evoke()
            self.send_response(200)
            self.send_header('Content-type', 'application/json')
            self.end_headers()
            response = {'evoked': success, 'status': controller_app.status}
            self.wfile.write(json.dumps(response).encode())

        elif self.path == '/control/hibernate':
            success = controller_app.hibernate()
            self.send_response(200)
            self.send_header('Content-type', 'application/json')
            self.end_headers()
            response = {'hibernated': success, 'status': controller_app.status}
            self.wfile.write(json.dumps(response).encode())
        else:
            self.send_response(404)
            self.end_headers()

    def do_GET(self):
        """Handle stats query - FIXED VERSION"""
        if self.path == '/stats/load':
            try:
                stats = controller_app.get_load_stats()
                self.send_response(200)
                self.send_header('Content-type', 'application/json')
                self.end_headers()
                self.wfile.write(json.dumps(stats).encode())
            except Exception as e:
                controller_app.logger.error(f"Error getting stats: {e}")
                self.send_response(500)
                self.end_headers()
        else:
            self.send_response(404)
            self.end_headers()

    def log_message(self, format, *args):
        """Suppress HTTP request logs"""
        pass

def run_server():
    try:
        server = HTTPServer(('127.0.0.1', controller_app.wsgi_port), UnifiedHandler)
        controller_app.logger.info(f"  REST API server started on port {controller_ap
        server.serve_forever()
    except Exception as e:
        controller_app.logger.error(f"  Failed to start REST API: {e}")
```

```
        # Start server in daemon thread
        thread = Thread(target=run_server, daemon=True)
        thread.start()
        controller_app.logger.info("REST API thread started")

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        """Handle switch connections"""
        datapath = ev.msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        self.datapaths[datapath.id] = datapath

        if self.status == 'active':
            self.logger.info(f"  Switch DPID={datapath.id} connected (ACTIVE mode)")
        else:
            self.logger.info(f"   Switch DPID={datapath.id} connected (PARKED mode)")

        # Install table-miss flow
        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                          ofproto.OFPCML_NO_BUFFER)]
        self.add_flow(datapath, 0, match, actions)

    def add_flow(self, datapath, priority, match, actions, buffer_id=None):
        """Install flow entry"""
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]

        if buffer_id:
            mod = parser.OFPFlowMod(datapath=datapath, buffer_id=buffer_id,
                                    priority=priority, match=match, instructions=inst)
        else:
            mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                                    match=match, instructions=inst)
        datapath.send_msg(mod)

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def _packet_in_handler(self, ev):
        """Handle packet-in events - ONLY when active"""

        # Don't process packets when parked
        if self.status != 'active':
```

```
        return

    self.packet_in_count += 1

    msg = ev.msg
    datapath = msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser
    in_port = msg.match['in_port']

    pkt = packet.Packet(msg.data)
    eth = pkt.get_protocols(ethernet.ethernet)[0]

    if eth.ethertype == ether_types.ETH_TYPE_LLDP:
        return

    dst = eth.dst
    src = eth.src
    dpid = datapath.id

    self.mac_to_port.setdefault(dpid, {})
    self.mac_to_port[dpid][src] = in_port

    if dst in self.mac_to_port[dpid]:
        out_port = self.mac_to_port[dpid][dst]
    else:
        out_port = ofproto.OFPP_FLOOD

    actions = [parser.OFPActionOutput(out_port)]

    # Install flow if not flooding
    if out_port != ofproto.OFPP_FLOOD:
        match = parser.OFPMatch(in_port=in_port, eth_dst=dst, eth_src=src)
        if msg.buffer_id != ofproto.OFP_NO_BUFFER:
            self.add_flow(datapath, 1, match, actions, msg.buffer_id)
            return
        else:
            self.add_flow(datapath, 1, match, actions)

    # Send packet out
    data = None
    if msg.buffer_id == ofproto.OFP_NO_BUFFER:
        data = msg.data

    out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                              in_port=in_port, actions=actions, data=data)
```

```
        datapath.send_msg(out)

    def evoke(self):
        """Activate this controller"""
        old_status = self.status
        self.status = 'active'
        self.packet_in_count = 0
        self.start_time = time.time()

        self.logger.info("="*50)
        self.logger.info("  CONTROLLER EVOKED!")
        self.logger.info(f"Status changed: {old_status} → {self.status}")
        self.logger.info(f"Ready to process traffic on port {self.wsgi_port}")
        self.logger.info(f"Currently managing {len(self.datapaths)} switches")
        self.logger.info("="*50)

        return True

    def hibernate(self):
        """Put controller back to sleep"""
        self.status = 'parked'
        self.datapaths.clear()
        self.mac_to_port.clear()
        self.packet_in_count = 0

        self.logger.info("  CONTROLLER HIBERNATED - Back to PARKED mode")
        return True

    def get_load_stats(self):
        """Return current load metrics - FIXED VERSION"""
        elapsed = time.time() - self.start_time

        # Prevent division by zero
        if elapsed < 0.001:
            elapsed = 0.001

        packet_in_rate = self.packet_in_count / elapsed

        return {
            'packet_in_rate': packet_in_rate,
            'total_packet_in': self.packet_in_count,
            'switch_count': len(self.datapaths),
            'status': self.status,
            'uptime': elapsed
        }
-e
```

File: ./controllers/master_controller.py

```python
from os_ken.base import app_manager
from os_ken.controller import ofp_event
from os_ken.controller.handler import set_ev_cls, MAIN_DISPATCHER
import requests
import json
import traceback
import threading
import time
import subprocess

class MasterController(app_manager.OSKenApp):

    def __init__(self, *args, **kwargs):
        super(MasterController, self).__init__(*args, **kwargs)

        self.slave_controllers = {
            'slave1': {
                'url': 'http://127.0.0.1:8081',
                'status': 'active',
                'port': 6634,
                'load': 0,
                'switches': ['s1', 's2', 's3', 's4']
            },
            'slave2': {
                'url': 'http://127.0.0.1:8082',
                'status': 'active',
                'port': 6635,
                'load': 0,
                'switches': ['s5', 's6', 's7', 's8']
            }
        }

        self.parked_controllers = {
            'parked1': {
                'url': 'http://127.0.0.1:8083',
                'status': 'parked',
                'port': 6636
            }
        }

        self.HIGH_LOAD_THRESHOLD = 4.0
        self.LOW_LOAD_THRESHOLD = 1.0
        self.running = True
```

```
        self.logger.info("="*50)
        self.logger.info("MASTER CONTROLLER STARTED")
        self.logger.info("With REAL OpenFlow Switch Migration")
        self.logger.info(f"High Load Threshold: {self.HIGH_LOAD_THRESHOLD} packets/sec")
        self.logger.info(f"Low Load Threshold: {self.LOW_LOAD_THRESHOLD} packets/sec")
        self.logger.info("="*50)

        self.monitor_thread = threading.Thread(target=self._monitor_loop, daemon=False)
        self.monitor_thread.start()
        self.logger.info("Monitoring thread started successfully")

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def _packet_in_handler(self, ev):
        pass

    def _monitor_loop(self):
        self.logger.info("Monitor loop beginning...\n")

        while self.running:
            try:
                self.logger.info("="*50)
                self.logger.info("--- Monitoring Cycle ---")
                self.logger.info("="*50)

                slave_items = list(self.slave_controllers.items())

                for slave_id, info in slave_items:
                    if info['status'] == 'active':
                        load = self._get_controller_load(info['url'])
                        info['load'] = load

                        switches_str = ', '.join(info.get('switches', []))
                        self.logger.info(f"{slave_id}: Load = {load:.2f} pkt/s | Switches: [

                        if load > self.HIGH_LOAD_THRESHOLD:
                            self.logger.warning(f"   {slave_id} OVERLOADED!")
                            self._handle_overload(slave_id)
                        elif load < self.LOW_LOAD_THRESHOLD:
                            self.logger.info(f"  {slave_id} underutilized")
                        else:
                            self.logger.info(f"  {slave_id} load is normal")

                self.logger.info("Sleeping for 10 seconds...\n")
                time.sleep(10)

            except Exception as e:
```

```
            self.logger.error(f"Error in monitoring cycle: {e}")
            self.logger.error(traceback.format_exc())
            time.sleep(10)

    def _get_controller_load(self, url):
        try:
            response = requests.get(f"{url}/stats/load", timeout=2)
            if response.status_code == 200:
                data = response.json()
                return data.get('packet_in_rate', 0)
            return 0
        except:
            return 0

    def _handle_overload(self, overloaded_slave_id):
        try:
            parked_items = list(self.parked_controllers.items())

            for parked_id, info in parked_items:
                if info['status'] == 'parked':
                    self.logger.info(f"  EVOKING {parked_id} to handle load!")
                    self._evoke_controller(parked_id, overloaded_slave_id)
                    return

            self.logger.warning("  No parked controllers available!")
            self._balance_between_active_controllers(overloaded_slave_id)

        except Exception as e:
            self.logger.error(f"Error handling overload: {e}")
            self.logger.error(traceback.format_exc())

    def _evoke_controller(self, parked_id, from_controller_id):
        try:
            url = self.parked_controllers[parked_id]['url']
            self.logger.info(f"   Sending evoke request to {url}/control/evoke")

            response = requests.post(f"{url}/control/evoke", timeout=5)

            if response.status_code == 200:
                controller_info = self.parked_controllers.pop(parked_id)
                controller_info['status'] = 'active'
                controller_info['load'] = 0
                controller_info['switches'] = []

                new_slave_id = f"slave{len(self.slave_controllers) + 1}"
                self.slave_controllers[new_slave_id] = controller_info
```

```
                self.logger.info(f"     {parked_id} is now {new_slave_id} (ACTIVE)")
                self.logger.info(f"     Starting REAL switch migration...")

                self._migrate_switches(from_controller_id, new_slave_id)

                for slave_id in ['slave1', 'slave2']:
                    if slave_id != from_controller_id:
                        if self.slave_controllers[slave_id]['load'] > self.HIGH_LOAD_THRESHOL
                            self._migrate_switches(slave_id, new_slave_id)

            else:
                self.logger.error(f"   Failed to evoke: HTTP {response.status_code}")

        except Exception as e:
            self.logger.error(f"   Failed to evoke {parked_id}: {e}")

    def _reconnect_switch_to_controller(self, switch_name, new_controller_port):
        """Reconnect switch to different controller via OpenFlow"""
        try:
            self.logger.info(f"      → Reconnecting {switch_name} to port {new_controller_por

            cmd = f"sudo ovs-vsctl set-controller {switch_name} tcp:127.0.0.1:{new_controller
            result = subprocess.run(cmd, shell=True, capture_output=True, text=True)

            if result.returncode == 0:
                self.logger.info(f"       {switch_name} reconnected successfully!")
                time.sleep(1)  # Give switch time to reconnect
                return True
            else:
                self.logger.error(f"       Failed: {result.stderr}")
                return False

        except Exception as e:
            self.logger.error(f"       Error: {e}")
            return False

    def _migrate_switches(self, from_id, to_id):
        """Migrate switches with REAL OpenFlow reconnection"""
        try:
            from_ctrl = self.slave_controllers.get(from_id)
            to_ctrl = self.slave_controllers.get(to_id)

            if not from_ctrl or not to_ctrl:
                return
```

```
            current_switches = from_ctrl.get('switches', [])
            if len(current_switches) <= 1:
                self.logger.info(f"      {from_id} has only {len(current_switches)} switch(es
                return

            num_to_migrate = len(current_switches) // 2
            switches_to_migrate = current_switches[-num_to_migrate:]

            self.logger.info(f"    Migrating {switches_to_migrate} from {from_id} to {to_id}

            # PERFORM REAL MIGRATION
            to_port = to_ctrl['port']
            migrated = []
            for switch in switches_to_migrate:
                if self._reconnect_switch_to_controller(switch, to_port):
                    migrated.append(switch)

            # Update records only for successfully migrated switches
            from_ctrl['switches'] = [s for s in current_switches if s not in migrated]
            to_ctrl['switches'] = to_ctrl.get('switches', []) + migrated

            self.logger.info(f"    Migrated {len(migrated)}/{len(switches_to_migrate)} switc
            self.logger.info(f"      {from_id}: {from_ctrl['switches']}")
            self.logger.info(f"      {to_id}: {to_ctrl['switches']}")

            return True

        except Exception as e:
            self.logger.error(f"   Migration error: {e}")
            return False

    def _balance_between_active_controllers(self, overloaded_id):
        self.logger.info(f"   Balancing between active controllers...")

        min_load = float('inf')
        min_loaded_id = None

        for slave_id, info in self.slave_controllers.items():
            if slave_id != overloaded_id and info['status'] == 'active':
                if info['load'] < min_load:
                    min_load = info['load']
                    min_loaded_id = slave_id

        if min_loaded_id:
            self.logger.info(f"   Found {min_loaded_id} with load {min_load:.2f}")
            self._migrate_switches(overloaded_id, min_loaded_id)
```

```
        else:
            self.logger.warning(f"   No suitable controller found")
-e
```

File: ./controllers/slave_controller.py

```
from os_ken.base import app_manager
from os_ken.controller import ofp_event
from os_ken.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER, set_ev_cls
from os_ken.ofproto import ofproto_v1_3
from os_ken.lib.packet import packet, ethernet, ether_types
import json
import time
import os
from threading import Thread
from http.server import HTTPServer, BaseHTTPRequestHandler

class SlaveController(app_manager.OSKenApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(SlaveController, self).__init__(*args, **kwargs)

        self.mac_to_port = {}
        self.datapaths = {}
        self.packet_in_count = 0
        self.start_time = time.time()
        self.status = 'active'
        self.wsgi_port = int(os.environ.get('WSGI_PORT', 8081))

        self.start_rest_api()

        self.logger.info("="*50)
        self.logger.info(f"SLAVE CONTROLLER on port {self.wsgi_port}")
        self.logger.info("="*50)

    def start_rest_api(self):
        controller_app = self

        class StatsHandler(BaseHTTPRequestHandler):
            def do_GET(self):
                if self.path == '/stats/load':
                    stats = controller_app.get_load_stats()
                    self.send_response(200)
                    self.send_header('Content-type', 'application/json')
                    self.end_headers()
```

```
                self.wfile.write(json.dumps(stats).encode())
            else:
                self.send_response(404)
                self.end_headers()

        def log_message(self, format, *args):
            pass

    def run_server():
        server = HTTPServer(('127.0.0.1', controller_app.wsgi_port), StatsHandler)
        server.serve_forever()

    thread = Thread(target=run_server, daemon=True)
    thread.start()

@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    self.datapaths[datapath.id] = datapath
    self.logger.info(f"Switch connected: DPID={datapath.id}")

    match = parser.OFPMatch()
    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                      ofproto.OFPCML_NO_BUFFER)]
    self.add_flow(datapath, 0, match, actions)

def add_flow(self, datapath, priority, match, actions, buffer_id=None):
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                         actions)]
    if buffer_id:
        mod = parser.OFPFlowMod(datapath=datapath, buffer_id=buffer_id,
                                priority=priority, match=match,
                                instructions=inst)
    else:
        mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                                match=match, instructions=inst)
    datapath.send_msg(mod)

@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def _packet_in_handler(self, ev):
```

```
        self.packet_in_count += 1

        msg = ev.msg
        datapath = msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        in_port = msg.match['in_port']

        pkt = packet.Packet(msg.data)
        eth = pkt.get_protocols(ethernet.ethernet)[0]

        if eth.ethertype == ether_types.ETH_TYPE_LLDP:
            return

        dst = eth.dst
        src = eth.src
        dpid = datapath.id

        self.mac_to_port.setdefault(dpid, {})
        self.mac_to_port[dpid][src] = in_port

        if dst in self.mac_to_port[dpid]:
            out_port = self.mac_to_port[dpid][dst]
        else:
            out_port = ofproto.OFPP_FLOOD

        actions = [parser.OFPActionOutput(out_port)]

        if out_port != ofproto.OFPP_FLOOD:
            match = parser.OFPMatch(in_port=in_port, eth_dst=dst, eth_src=src)
            if msg.buffer_id != ofproto.OFP_NO_BUFFER:
                self.add_flow(datapath, 1, match, actions, msg.buffer_id)
                return
            else:
                self.add_flow(datapath, 1, match, actions)

        data = None
        if msg.buffer_id == ofproto.OFP_NO_BUFFER:
            data = msg.data

        out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                                  in_port=in_port, actions=actions, data=data)
        datapath.send_msg(out)

    def get_load_stats(self):
        elapsed = time.time() - self.start_time
```

```
        packet_in_rate = self.packet_in_count / elapsed if elapsed > 0 else 0

        return {
            'packet_in_rate': packet_in_rate,
            'total_packet_in': self.packet_in_count,
            'switch_count': len(self.datapaths),
            'status': self.status,
            'uptime': elapsed
        }
-e
```

File: ./utils/traffic_loader.py

```
#!/usr/bin/env python3
"""
Traffic Data Loader for CIC-Bell-DNS 2021 Dataset
Processes real traffic traces for SDN simulation
"""
import pandas as pd
import numpy as np
import os
import glob

class TrafficLoader:
    """Load and process real traffic data from CIC-Bell-DNS 2021"""

    def __init__(self, traffic_dir="../data/traffic"):
        self.traffic_dir = traffic_dir
        self.traffic_data = None
        self.hourly_pattern = None

    def load_cic_dns_2021(self, day='FirstDayBenign'):
        """
        Load CIC-Bell-DNS 2021 dataset

        Args:
            day: Which day to load ('FirstDayBenign', 'SecondDay', 'ThirdDay')

        Returns:
            Traffic statistics
        """
        # Try to find CSV files in the extracted directory
        day_dir = os.path.join(self.traffic_dir, day)

        if os.path.exists(day_dir):
            csv_files = glob.glob(os.path.join(day_dir, '*.csv'))
```

```
            if csv_files:
                print(f"  Found {len(csv_files)} CSV files in {day}")

                try:
                    # Try to load with error handling for malformed CSV
                    print(f"  Attempting to load: {os.path.basename(csv_files[0])}")

                    # Load with error tolerance
                    df = pd.read_csv(
                        csv_files[0],
                        nrows=100000,
                        on_bad_lines='skip',  # Skip malformed lines
                        low_memory=False,
                        encoding='utf-8',
                        encoding_errors='ignore'
                    )

                    print(f"  Loaded {len(df)} traffic records")
                    self.traffic_data = self._process_real_data(df)

                except Exception as e:
                    print(f"   Error loading CSV: {e}")
                    print(f"   Falling back to synthetic traffic data")
                    self.traffic_data = self._generate_synthetic_traffic()
            else:
                print(f"   No CSV files found in {day_dir}, using synthetic data")
                self.traffic_data = self._generate_synthetic_traffic()
        else:
            print(f"   Directory {day_dir} not found, using synthetic data")
            self.traffic_data = self._generate_synthetic_traffic()

        return self.traffic_data

    def _process_real_data(self, df):
        """Process real CIC-DNS-2021 data"""
        print("  Processing real traffic data...")
        print(f"   Columns available: {len(df.columns)}")

        # Print first few column names to understand the data
        print(f"   Sample columns: {list(df.columns[:5])}")

        # CIC-DNS-2021 columns vary, try common ones
        time_columns = [
            'Timestamp', 'timestamp', 'Time', 'time',
            'Flow Start Time', 'flow_start_time'
```

```
]
time_col = None

for col in time_columns:
    if col in df.columns:
        time_col = col
        break

if time_col:
    try:
        # Try to parse timestamps
        df['Hour'] = pd.to_datetime(df[time_col], errors='coerce').dt.hour

        # Remove rows where hour parsing failed
        df = df.dropna(subset=['Hour'])

        if len(df) > 0:
            hourly_flows = df.groupby('Hour').size().to_dict()

            # Normalize to 0-1 range
            max_flows = max(hourly_flows.values()) if hourly_flows else 1
            normalized_flows = {h: (hourly_flows.get(h, 0) / max_flows)
                                for h in range(24)}

            print(f"  Extracted real hourly traffic patterns from {len(df)} records")
            print(f"   Hours covered: {sorted(hourly_flows.keys())}")

            return {
                'hourly_pattern': normalized_flows,
                'total_flows': len(df),
                'source': 'CIC-Bell-DNS-2021 (Real Data)',
                'hours_with_data': len(hourly_flows)
            }
        else:
            print(f"   No valid timestamp data after parsing")
            return self._generate_synthetic_traffic()

    except Exception as e:
        print(f"   Could not parse timestamps: {e}")
        return self._generate_synthetic_traffic()
else:
    # No timestamp column - analyze flow counts directly
    print(f"   No timestamp column found")
    print(f"   Available columns: {list(df.columns[:10])}")

    # If we have flow data, create a simple pattern based on row distribution
```

```
        if len(df) > 1000:
            print(f"  Using flow-based traffic estimation from {len(df)} records")
            # Divide dataset into 24 hour bins based on row position
            rows_per_hour = len(df) // 24
            hourly_pattern = {}

            for hour in range(24):
                start_idx = hour * rows_per_hour
                end_idx = (hour + 1) * rows_per_hour if hour < 23 else len(df)
                flows_in_hour = end_idx - start_idx
                hourly_pattern[hour] = flows_in_hour

            # Normalize
            max_flows = max(hourly_pattern.values())
            normalized = {h: (v / max_flows) for h, v in hourly_pattern.items()}

            return {
                'hourly_pattern': normalized,
                'total_flows': len(df),
                'source': 'CIC-Bell-DNS-2021 (Flow-based estimation)',
                'hours_with_data': 24
            }
        else:
            return self._generate_synthetic_traffic()

    def _generate_synthetic_traffic(self):
        """
        Generate synthetic traffic matching CIC-DNS-2021 patterns
        Based on typical enterprise network behavior from the paper
        """
        print("  Using synthetic traffic (CIC-DNS-2021 patterns)")

        # Realistic hourly pattern (from paper analysis and typical enterprise networks)
        hourly_pattern = {
            0: 0.15,   # Night - 15% of peak
            1: 0.12,
            2: 0.10,   # Lowest traffic
            3: 0.11,
            4: 0.13,
            5: 0.20,   # Early morning
            6: 0.40,   # Morning rise
            7: 0.65,
            8: 0.90,   # Work starts
            9: 1.00,   # Peak morning (100%)
            10: 0.95,
            11: 0.92,
```

```
        12: 0.85,  # Lunch dip
        13: 0.88,
        14: 0.95,  # Afternoon peak
        15: 0.92,
        16: 0.88,
        17: 0.75,  # End of day
        18: 0.55,
        19: 0.40,  # Evening
        20: 0.35,
        21: 0.28,
        22: 0.22,  # Night
        23: 0.18
    }

    return {
        'hourly_pattern': hourly_pattern,
        'total_flows': 1000000,  # 1M flows per day
        'packet_in_ratio': 0.001,  # 0.1% trigger controller (from paper)
        'source': 'Synthetic (CIC-DNS-2021 based)',
        'hours_with_data': 24
    }

def get_traffic_multiplier(self, hour):
    """
    Get traffic multiplier for given hour

    Args:
        hour: Hour of day (0-23)

    Returns:
        Traffic multiplier (0.0-1.0)
    """
    if self.traffic_data is None:
        self.load_cic_dns_2021()

    return self.traffic_data['hourly_pattern'].get(hour, 0.5)

def get_hourly_flows(self):
    """Get complete hourly flow pattern"""
    if self.traffic_data is None:
        self.load_cic_dns_2021()

    return self.traffic_data['hourly_pattern']

def print_pattern_summary(self):
    """Print traffic pattern summary"""
```

```
        if self.traffic_data is None:
            self.load_cic_dns_2021()

        print(f"\n  Traffic Pattern Summary:")
        print(f"  Source: {self.traffic_data.get('source', 'Unknown')}")
        print(f"  Total Flows: {self.traffic_data.get('total_flows', 'N/A'):,}")
        print(f"  Hours with Data: {self.traffic_data.get('hours_with_data', 'N/A')}")
        print(f"\n  Hourly Traffic Pattern:")

        pattern = self.traffic_data['hourly_pattern']

        # Show sample hours
        print(f"    Peak hours:")
        peak_hours = sorted(pattern.items(), key=lambda x: x[1], reverse=True)[:5]
        for hour, multiplier in peak_hours:
            print(f"      {hour:02d}:00 - {multiplier:.2f}x")

        print(f"\n    Low hours:")
        low_hours = sorted(pattern.items(), key=lambda x: x[1])[:5]
        for hour, multiplier in low_hours:
            print(f"      {hour:02d}:00 - {multiplier:.2f}x")

        # Period averages
        print(f"\n  Period Averages:")
        for period, hours in [('Night (00-06)', range(0, 6)),
                              ('Morning (06-12)', range(6, 12)),
                              ('Afternoon (12-18)', range(12, 18)),
                              ('Evening (18-24)', range(18, 24))]:
            avg = np.mean([pattern[h] for h in hours])
            print(f"    {period}: {avg:.2f}x average")


def test_traffic_loader():
    """Test the traffic loader"""
    loader = TrafficLoader()

    # Try to load real data
    print("="*60)
    print("Testing Traffic Loader with Real CIC-DNS-2021 Data")
    print("="*60 + "\n")

    traffic_data = loader.load_cic_dns_2021('FirstDayBenign')

    # Print summary
    loader.print_pattern_summary()
```

```
    print(f"\n  Sample Traffic Multipliers:")
    for hour in [2, 9, 14, 22]:
        multiplier = loader.get_traffic_multiplier(hour)
        print(f"  {hour:02d}:00 - {multiplier:.2f}x")

    print(f"\n  Traffic loader test completed!")


if __name__ == "__main__":
    test_traffic_loader()
-e
```

File: ./utils/topology_loader.py

```
#!/usr/bin/env python3
"""
Topology Loader for Real Network Data
Loads and processes Internet Topology Zoo graphs
"""
import networkx as nx
import numpy as np
import os

class TopologyLoader:
    """Load and process real network topologies"""

    def __init__(self, topology_dir="../data/topologies"):
        self.topology_dir = topology_dir
        self.topologies = {
            'gridnet': 'Gridnet.gml',
            'bellcanada': 'BellCanada.gml',
            'os3e': 'Os3e.gml'
        }

    def load_topology(self, name='os3e'):
        """
        Load a topology from file

        Args:
            name: Topology name (gridnet, bellcanada, os3e)

        Returns:
            Graph object with nodes and edges
        """
        if name.lower() not in self.topologies:
            raise ValueError(f"Unknown topology: {name}. Choose from {list(self.topologies.ke
```

```
        filepath = os.path.join(self.topology_dir, self.topologies[name.lower()])

        if not os.path.exists(filepath):
            raise FileNotFoundError(f"Topology file not found: {filepath}\nCurrent dir: {os.g

        # Load GML file
        G = nx.read_gml(filepath, label='id')

        print(f"  Loaded {name}: {G.number_of_nodes()} nodes, {G.number_of_edges()} edges")

        return G

    def calculate_distance_matrix(self, G):
        """
        Calculate shortest path distances between all nodes

        Returns:
            Distance matrix (N x N numpy array)
        """
        nodes = list(G.nodes())
        n = len(nodes)
        distance_matrix = np.zeros((n, n))

        # Calculate all shortest paths
        shortest_paths = dict(nx.all_pairs_shortest_path_length(G))

        for i, node_i in enumerate(nodes):
            for j, node_j in enumerate(nodes):
                if i == j:
                    distance_matrix[i][j] = 0
                else:
                    distance_matrix[i][j] = shortest_paths[node_i].get(node_j, float('inf'))

        return distance_matrix, nodes

    def get_topology_info(self, name='os3e'):
        """
        Get comprehensive topology information

        Returns:
            dict with topology statistics
        """
        G = self.load_topology(name)
        distance_matrix, nodes = self.calculate_distance_matrix(G)
```

```
        info = {
            'name': name,
            'num_nodes': G.number_of_nodes(),
            'num_edges': G.number_of_edges(),
            'nodes': nodes,
            'distance_matrix': distance_matrix,
            'avg_degree': np.mean([d for n, d in G.degree()]),
            'diameter': nx.diameter(G) if nx.is_connected(G) else float('inf')
        }

        return info


def test_topology_loader():
    """Test the topology loader"""
    loader = TopologyLoader()

    # Test all topologies
    for topo_name in ['gridnet', 'bellcanada', 'os3e']:
        try:
            info = loader.get_topology_info(topo_name)
            print(f"\n  {topo_name.upper()} Topology:")
            print(f"  Nodes: {info['num_nodes']}")
            print(f"  Edges: {info['num_edges']}")
            print(f"  Avg Degree: {info['avg_degree']:.2f}")
            print(f"  Diameter: {info['diameter']}")
        except Exception as e:
            print(f"\n  Error loading {topo_name}: {e}")


if __name__ == "__main__":
    test_topology_loader()
-e
```

File: ./slave_controller.py

```
from os_ken.base import app_manager
from os_ken.controller import ofp_event
from os_ken.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER
from os_ken.controller.handler import set_ev_cls
from os_ken.ofproto import ofproto_v1_3
from os_ken.lib.packet import packet, ethernet, ether_types
from os_ken.app.wsgi import WSGIApplication, ControllerBase, route
from webob import Response
import json
import time
```

```python
import sys

slave_instance_name = 'slave_api_app'

class SlaveController(app_manager.OSKenApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
    _CONTEXTS = {'wsgi': WSGIApplication}

    def __init__(self, *args, **kwargs):
        super(SlaveController, self).__init__(*args, **kwargs)

        # Determine which port to use based on command line
        # Check if port was specified
        wsgi_port = 8081  # Default

        # You can pass port via environment variable
        import os
        if 'WSGI_PORT' in os.environ:
            wsgi_port = int(os.environ['WSGI_PORT'])

        # Configure WSGI
        wsgi = kwargs['wsgi']
        wsgi.start(
            host='127.0.0.1',
            port=wsgi_port
        )

        # State tracking
        self.mac_to_port = {}
        self.datapaths = {}
        self.packet_in_count = 0
        self.start_time = time.time()
        self.status = 'active'

        # Register REST API
        wsgi.register(SlaveStatsController, {slave_instance_name: self})

        self.logger.info("="*50)
        self.logger.info(f"SLAVE CONTROLLER STARTED (ACTIVE MODE) on port {wsgi_port}")
        self.logger.info("="*50)

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        """Handle new switch connection"""
        datapath = ev.msg.datapath
        ofproto = datapath.ofproto
```

```
        parser = datapath.ofproto_parser

        self.datapaths[datapath.id] = datapath
        self.logger.info(f"Switch connected: DPID={datapath.id}")

        # Install table-miss flow entry
        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                          ofproto.OFPCML_NO_BUFFER)]
        self.add_flow(datapath, 0, match, actions)

    def add_flow(self, datapath, priority, match, actions, buffer_id=None):
        """Add a flow entry to the switch"""
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,
                                             actions)]
        if buffer_id:
            mod = parser.OFPFlowMod(datapath=datapath, buffer_id=buffer_id,
                                    priority=priority, match=match,
                                    instructions=inst)
        else:
            mod = parser.OFPFlowMod(datapath=datapath, priority=priority,
                                    match=match, instructions=inst)
        datapath.send_msg(mod)

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def _packet_in_handler(self, ev):
        """Handle packet-in events"""
        self.packet_in_count += 1

        msg = ev.msg
        datapath = msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        in_port = msg.match['in_port']

        pkt = packet.Packet(msg.data)
        eth = pkt.get_protocols(ethernet.ethernet)[0]

        if eth.ethertype == ether_types.ETH_TYPE_LLDP:
            return

        dst = eth.dst
        src = eth.src
```

```
        dpid = datapath.id

        self.mac_to_port.setdefault(dpid, {})
        self.mac_to_port[dpid][src] = in_port

        if dst in self.mac_to_port[dpid]:
            out_port = self.mac_to_port[dpid][dst]
        else:
            out_port = ofproto.OFPP_FLOOD

        actions = [parser.OFPActionOutput(out_port)]

        if out_port != ofproto.OFPP_FLOOD:
            match = parser.OFPMatch(in_port=in_port, eth_dst=dst, eth_src=src)
            if msg.buffer_id != ofproto.OFP_NO_BUFFER:
                self.add_flow(datapath, 1, match, actions, msg.buffer_id)
                return
            else:
                self.add_flow(datapath, 1, match, actions)

        data = None
        if msg.buffer_id == ofproto.OFP_NO_BUFFER:
            data = msg.data

        out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                                  in_port=in_port, actions=actions, data=data)
        datapath.send_msg(out)

    def get_load_stats(self):
        """Return current load metrics"""
        elapsed = time.time() - self.start_time
        packet_in_rate = self.packet_in_count / elapsed if elapsed > 0 else 0

        return {
            'packet_in_rate': packet_in_rate,
            'total_packet_in': self.packet_in_count,
            'switch_count': len(self.datapaths),
            'status': self.status,
            'uptime': elapsed
        }


class SlaveStatsController(ControllerBase):
    def __init__(self, req, link, data, **config):
        super(SlaveStatsController, self).__init__(req, link, data, **config)
        self.slave_app = data[slave_instance_name]
```

```
    @route('stats', '/stats/load', methods=['GET'])
    def get_load(self, req, **kwargs):
        """REST API endpoint to query controller load"""
        stats = self.slave_app.get_load_stats()
        body = json.dumps(stats)
        return Response(content_type='application/json', body=body)
-e
```